



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

REINFORCED LEARNING OF AGENTS WITH MEMORY

POSILOVANÉ UČENÍ AGENTŮ S PAMĚTÍ

MASTER'S THESIS

DIPLOMOVÁ PRÁCE

AUTHOR

AUTOR PRÁCE

Bc. SAMUEL KUČHTA

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. MICHAL HRADIŠ, Ph.D.

BRNO 2026

Master's Thesis Assignment



173497

Institut: Department of Computer Graphics and Multimedia (DCGM)
Student: **Kuchta Samuel, Bc.**
Programme: Information Technology and Artificial Intelligence
Specialization: Computer Vision
Title: **Reinforcement Learning of Agents with Memory**
Category: Artificial Intelligence
Academic year: 2025/26

Assignment:

1. Study the theory and algorithms of reinforcement learning.
2. Review current state-of-the-art algorithms for training agents that use memory.
3. Propose methods, their extensions, environments, and an experimental structure suitable for gaining new insights into the learning of agents with memory.
4. Prepare the experimental environment and implement the selected method extensions.
5. Evaluate the behavior of the selected methods in experiments.
6. Assess the achieved results and discuss possible directions for future development.
7. Create a short video presenting your work, its goals, and its results.

Literature:

- Lauri, Hsu and Pajarinen: Partially Observable Markov Decision Processes in Robotics: A Survey, IEEE TRANSACTIONS ON ROBOTICS, 2023.
- Echchahed and Castro: A Survey of State Representation Learning for Deep Reinforcement Learning, Transactions on Machine Learning Research, 2025.
- Meng, Gorbet and Kulić: Memory-based Deep Reinforcement Learning for POMDPs, IROS, 2021.

Requirements for the semestral defence:
Goals 1-3, partial 4.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Hradiš Michal, Ing., Ph.D.**
Head of Department: Černocký Jan, prof. Dr. Ing.
Beginning of work: 1.11.2025
Submission deadline: 27.5.2026
Approval date: 12.5.2026

Abstract

This thesis contributes a purpose-built environment, **GridMazeWorld**, designed to confront reinforcement learning agents with multiple, interdependent memory demands under partial observability. The environment combines procedurally generated mazes, non-local button-door dependencies, periodic dynamics, regrowing resources, and an energy budget, requiring simultaneous spatial, sequential, and relational memory. Alongside a comprehensive survey of memory-focused benchmarks and mechanisms, I implement and compare recurrent (LSTM), and attention-based (Transformer) architectures under controlled training with curriculum management. My empirical study examines the influence of hyperparameter choices, network scaling, grid size, and curriculum design on the agents' ability to form and exploit internal memory, offering a systematic analysis of memory in reinforcement learning under partial observability.

Abstrakt

Táto diplomová práca predstavuje účelovo navrhnuté prostredie **GridMazeWorld**, vytvorené na konfrontáciu agentov posilňovaného učenia s viacerými vzájomne prepojenými pamäťovými nárokmi pri čiastočnej pozorovateľnosti. Prostredie kombinuje procedurálne generované bludiská, nelokálne závislosti medzi tlačidlami a dverami, periodickú dynamiku, obnovujúce sa zdroje a energetický rozpočet, čím si vyžaduje súčasné využívanie priestorovej, sekvenčnej a relačnej pamäte. Popri komplexnom prehľade benchmarkov a mechanizmov zameraných na pamäť implementujem a porovnávam rekurentné architektúry (LSTM) a architektúry založené na pozornosti (Transformer) v rámci kontrolovaného tréningu s riadením kurikula. Moja empirická štúdia skúma vplyv voľby hyperparametrov, škálovania sietí, veľkosti mriežky a návrhu kurikula na schopnosť agentov vytvárať a využívať internú pamäť, pričom ponúka systematickú analýzu pamäte v posilňovanom učení pri čiastočnej pozorovateľnosti.

Keywords

partial observability, reinforcement learning, memory, GridMazeWorld, recurrent neural networks.

Klíčové slová

částečná pozorovateľnosť, posilované učenie, pamäť, GridMazeWorld, rekurentní neuronové siete.

Reference

KUCHTA, Samuel. *Reinforced learning of agents with memory*. Brno, 2026. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Michal Hradiš, Ph.D.

Rozšírený abstrakt

Táto diplomová práca sa zaoberá mechanizmami pamäti v posilňovanom učení (reinforcement learning, RL) za podmienky čiastočnej pozorovateľnosti. Zatiaľ čo klasické metódy RL predpokladajú, že agent má prístup k úplnému stavu prostredia, reálne úlohy túto idealizáciu porušujú – senzory poskytujú len čiastočné informácie, kritické premenné zostávajú skryté. V takýchto prostrediach optimálne správanie závisí na celej histórii interakcií, čo robí pamäť základným predpokladom pre robustných agentov.

Hlavným prínosom práce je prostredie **GridMazeWorld** – rýchle a konfigurovateľné 2D mriežkové prostredie, ktoré kombinuje procedurálne generovanie bludísk, navigáciu, závislosti dverí a tlačidiel, periodické časovače, zdroje potravy a energetické obmedzenia. Agent vidí iba svoje okolie 3×3 bunky, zatiaľ čo mnohé informácie zostávajú skryté: priradenie tlačidiel k dverám, stav tlačidla (funkčné/rozbité), časovače dverí a regenerácie potravy, ani presná hodnota energie nie je známa. Aby agent obstál, musí si budovať internú reprezentáciu sveta, pamätať si históriu stlačení tlačidiel, predvídať periodické otváranie dverí a plánovať zber potravy pod hrozbou vyčerpania energie.

Procedurálne generovanie zaručuje, že bludisko zostáva vždy spojené a dvere sú umiestnené na kľúčových miestach, ktoré rozdeľujú priestor. Tlačidlá sú rozmiestnené do oddelených častí bludiska, čím vznikajú netriviálne závislosti medzi vzdialenými oblasťami. Regenerácia potravy prebieha s variabilným oneskorením.

Druhým hlavným prínosom je **dynamický správca kurikula**, ktorý automaticky prispôbuje obtiažnosť prostredia na základe nedávnej výkonnosti agenta. Obtiažnosť je riadená spojeným parametrom *complexity_level* (0–1) a diskretnou triedou úlohy (*basic* – navigácia a jedlo, *doors* – periodické automatické dvere, *buttons* – dvere ovládané tlačidlami, *complex* – mix oboch typov). Správca udržiava kľzavé okno odmien, vypočítava skóre voči najlepšej dosiahnutej odmene v danej fáze a podľa prahových hodnôt zvyšuje alebo znižuje zložitosť. Pri stagnácii prechádza medzi fázami (*basic* → *doors* → *buttons* → *complex*) a po prvom cykle volí ďalšiu fázu pravdepodobnostne podľa aktuálnej zložitosti a frekvencie návštev. Tento mechanizmus odstraňuje potrebu ručného plánovania kurikula.

Pre experimenty boli implementované dve architektúry agentov – LSTM a Transformer. Oba typy zdieľajú rovnaký vstup (10 diskretných tokenov). Okrem hlavnej RL straty (REINFORCE s baseline, resp. PPO) je možné pridať pomocné predikčné úlohy (regresia nasledujúcej energie a nasledujúceho pozorovania), ktoré podporujú utváranie latentnej reprezentácie dynamiky prostredia.

Kľúčové zistenia: Dynamické kurikulum a pridanie pomocných predikčných úloh výrazne zlepšilo výkon. Architektúra LSTM dosahovala lepšie výsledky oproti architektúre Transformer. PPO dosahovalo lepšie výsledky oproti REINFORCE s baseline. hyperparametre výrazne ovplyvňujú priebeh tréningu.

GridMazeWorld vyplní medzeru medzi jednoduchými izolovanými testami pamäti a výpočtovo náročnými 3D simulátormi. Prináša výpočtovo efektívne, čiastočne pozorovateľné prostredie, ktoré integruje viacero pamäťových nárokov súčasne, a spolu s dynamickým kurikulumom umožňuje reprodukovateľný a automatizovaný výskum pamäťových mechanizmov v posilňovanom učení.

Reinforced learning of agents with memory

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Mr Ing. Michal Hradiš Ph.D. I have listed all the literary sources, publications, and other sources used during the preparation of this thesis.

.....
Samuel Kuchta
May 26, 2026

Acknowledgements

My sincere gratitude goes to my supervisor, Ing. Michal Hradiš Ph.D., who has always been more than willing to provide me with guidance, feedback and materials crucial to writing this master's thesis. I also wish to thank my family for their continuous support.

Contents

1	Introduction	3
2	Reinforcement Learning Basics	4
3	Need for Memory in POMDPs	7
3.1	Memory mechanisms in reinforcement learning	7
3.2	Explicit memory mechanisms	9
3.3	Recent advances in memory mechanisms	10
3.4	Practical implications, applications, and outlook	13
4	State Representation Learning	14
4.1	Motivation and problem setting	14
4.2	Classical approaches to state representation	15
4.3	Neural state representation learning in model-free RL	15
4.4	Representation learning in model-based reinforcement learning	16
4.5	State representation learning under partial observability	16
4.6	Evaluation of learned state representations	17
4.7	State-of-the-art methods and current trends	17
5	Environments for Reinforcement Learning with Memory	18
5.1	Memory Modalities in Benchmark Design	19
5.2	Classic and Simple Benchmarks	20
5.3	Grid-World and 2D Environments	21
5.4	Complex 3D Environments	22
5.5	Comparative Analysis	26
5.6	Limitations of Current Benchmarks	28
6	Environment Design and Implementation	29
6.1	Design Principles	29
6.2	Observation and action spaces	30
6.3	Environment construction	31
6.4	Temporal dynamics and resource management	32
6.5	API and configurability	32
6.6	Computational efficiency and used algorithms	33
6.7	Conclusion	36
7	Agent Architecture and Training Algorithms	38
7.1	Agent network architectures	38

7.2	Training algorithms	41
7.3	Dynamic complexity and curriculum learning	42
7.4	System implementation and data flow	43
8	Methods and Experimental Protocol	45
8.1	Training algorithm and hyperparameter sweeps	45
8.2	Dynamic complexity parameters	47
8.3	Reproducibility	47
9	Results	48
9.1	Algorithm Comparison: REINFORCE vs. PPO	48
9.2	PPO Hyperparameter Tuning	49
9.3	Scaling to Larger Grids	52
9.4	Network Architecture Comparison	54
9.5	Task Curriculum Comparison	56
10	Future Work and Extensions	59
10.1	Environment Enhancements	59
10.2	Architectures and State Representation Learning	61
10.3	Training Algorithms and Curriculum Design	62
10.4	Evaluation, Diagnostics, and Reproducibility	62
10.5	Real-World Transfer	63
11	Conclusion	64
11.1	Key Findings	65
11.2	Final Remarks	65
	Bibliography	66

Chapter 1

Introduction

Reinforcement learning has achieved remarkable results in games, robotics, and control, yet most of these successes rely on the assumption of full observability: the agent receives a complete, noise-free description of the environment’s state. Real-world tasks rarely satisfy this ideal, as sensors provide only partial or noisy views, critical variables remain hidden, and decisions must be made under uncertainty. In such partially observable settings, optimal behaviour depends not only on the latest observation but on the entire history of past interactions, making memory a fundamental requirement for RL agents deployed in realistic environments [49, 29].

Partial observability is formally captured by Partially Observable Markov Decision Processes (POMDPs) [28]. In a POMDP, the agent must infer an information state – a summary of the history – that is sufficient for decision making. Exact information state tracking is usually intractable, so practical agents rely on approximate memory mechanisms such as recurrent neural networks, attention over past observations, or explicit memory buffers [24, 14, 51, 39]. Designing and evaluating such mechanisms is a central challenge for robust RL in complex domains.

A large body of research has addressed memory in RL, producing a variety of benchmarks and architectures. Simple tasks like the T-maze probe a single delayed cue, while more complex simulators, such as Obstacle Tower or NetHack, integrate multiple memory demands at a higher computational cost [27, 32]. Recent surveys and specialised benchmarks, including POPGym, MemoryGym, and POBAX, have formalised the notion of memory-improvability and provided controlled testbeds [36, 41, 50]. However, existing environments often isolate a single memory modality – spatial, sequential, relational, or temporal – and rarely combine them into one integrated challenge. A gap remains for a computationally light environment that simultaneously demands spatial mapping, relational reasoning, temporal prediction, and resource management, while supporting curriculum learning and reproducible experimentation.

This thesis investigates memory mechanisms in reinforcement learning under partial observability through a procedurally generated environment that combines multiple interdependent memory demands. It examines how curriculum design, auxiliary self-supervised objectives, and hyperparameter choices affect reinforcement learning with internal memory.

Its main contributions are GridMazeWorld, a fast and configurable 2D grid world featuring procedural maze generation, navigation, door–button dependencies, periodic timers, food sources, and energy constraints under partial observability; and a dynamic curriculum manager that automatically adapts task complexity based on the agent’s recent performance, enabling progressive learning without manual scheduling.

Chapter 2

Reinforcement Learning Basics

Reinforcement learning studies how an agent should act in a sequential decision problem to maximise long-term performance. At its core is the **agent-environment loop**: at each time step, the agent receives information from the environment, selects an action, and receives a scalar reward as the environment transitions. This closed feedback loop makes decisions consequential and history-dependent; the agent improves its policy by using observed transitions and rewards to guide future choices [49, 29].

The quantity that the agent seeks to maximise is the **return**. In the standard infinite-horizon, discounted formulation, the return from time t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (2.1)$$

where r_{t+k+1} is the reward at time $t+k+1$ and $\gamma \in [0, 1)$ is the **discount factor**. The discount factor both enforces convergence of the sum in continuing tasks and encodes a preference for more recent rewards; different problem formulations (finite horizon, average-reward) retain the same objective principle: choose policies that maximise expected long-term returns [49].

A core algorithmic challenge is the **exploration-exploitation trade-off**. At each decision point, the agent may **exploit**—selecting the action currently believed to yield the highest reward—or **explore**—trying actions with uncertain outcomes to gather information that could improve future performance. Practical algorithms manage this trade-off explicitly (e.g., ϵ -greedy, Upper Confidence Bound, or Thompson sampling) or implicitly (e.g., intrinsic motivation, optimism under uncertainty). Balancing exploration and exploitation is essential: insufficient exploration can trap an agent in suboptimal policies, while excessive exploration reduces empirical reward during learning [29].

When the system is fully observable, the canonical mathematical model is the **Markov Decision Process (MDP)**. An MDP is specified by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s' | s, a)$ the transition probabilities, $R(s, a)$ the immediate reward model, and γ the discount factor. The defining assumption—the **Markov property**—states that the distribution of the next state and reward depends only on the current state and action, so that the current state is a sufficient statistic for future evolution [49, 29].

Solving MDPs relies on **value functions** that quantify long-term quality. For a policy π , the state-value function

$$V^\pi(s) = \mathbb{E}[G_t \mid s_t = s, \pi], \quad (2.2)$$

and the action-value function

$$Q^\pi(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a, \pi], \quad (2.3)$$

satisfy the recursive **Bellman equations**. The Bellman expectation equation expresses the value of a state in terms of immediate reward and the discounted value of successor states; replacing the policy expectation with a maximisation yields the Bellman optimality equations, which underpin dynamic programming, temporal-difference learning, and actor-critic methods [49].

Many real-world problems violate full observability: sensors are noisy, parts of the environment are hidden, or agents have limited bandwidth. Such problems are modelled as **Partially Observable Markov Decision Processes (POMDPs)**. In a POMDP, the environment evolves in an underlying (hidden) state s , while the agent receives observations o sampled from an observation model $O(o \mid s, a)$, illustrated in Figure 2.1. The observations provide partial or noisy information about the actual state [28, 40].

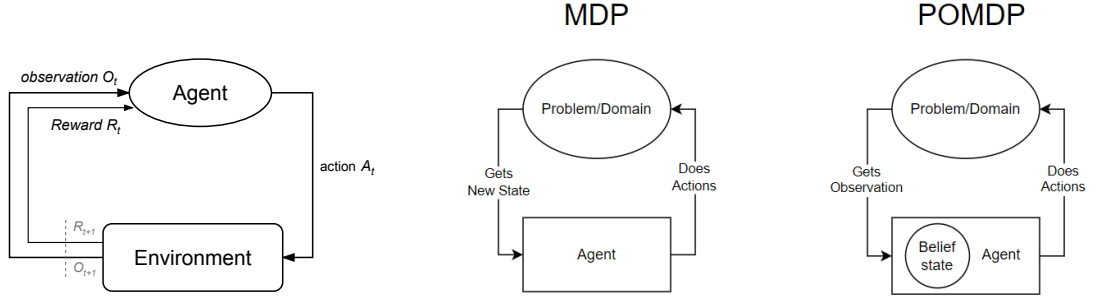


Figure 2.1: Agent in POMDP receiving observations instead of the whole state. [9]

To reason effectively under partial observability, the agent maintains an **information state**—a probability distribution over the underlying states conditioned on the agent’s history of actions and observations. The information state (a modern, descriptive substitute for the classical term “belief”) is a sufficient statistic for decision making and is updated by Bayesian filtering after each action–observation pair. For discrete state spaces, the Bayesian update is

$$b_{t+1}(s') = \eta O(o_{t+1} \mid s', a_t) \sum_{s \in \mathcal{S}} P(s' \mid s, a_t) b_t(s), \quad (2.4)$$

where η normalizes the distribution. By consolidating history into a single distribution, the information-state perspective enables the application of Bellman-style reasoning to partially observed problems [28].

A practical computational approach is to lift the POMDP into the **information-state MDP** by treating each information state b as a fully observable state: actions induce Bayesian updates between information states, and the immediate reward of a (information state, action) pair is the expectation of the underlying reward under b . Although this transformation renders the problem formally fully observable, it produces a continuous,

high-dimensional state space (the simplex of probability distributions) that is challenging to handle exactly. Notably, in finite-horizon POMDPs, the optimal value over information states is **piecewise-linear and convex**, enabling exact solution methods represented by a finite set of vectors; however, exact dynamic programming scales poorly in practice [28, 40].

Consequently, modern POMDP methods concentrate on scalable approximations and online planning. Representative strategies include **point-based value iteration**—which selectively updates value at a representative set of reachable information states—**sampling-based and online search** methods (e.g., Monte Carlo tree search from the current information state), and **function approximation** approaches that learn compact parametric value or policy representations [40]. These methods focus computation on relevant regions of information space and combine principled Bayesian filtering with pragmatic approximations to manage the dual challenges of uncertainty and dimensionality.

Taken together, the literature organises sequential decision problems along two linked axes: how the agent quantifies long-run performance (returns, value functions, Bellman recursions) and how much information the agent has about the environment (full vs partial observability). The **information state** perspective provides an operational and modern way to handle partial observability by consolidating history into a single sufficient statistic; the principal engineering challenge is therefore designing scalable representations and approximate planning or learning algorithms that work well under uncertainty and limited data [49, 28, 40, 29].

Chapter 3

Need for Memory in POMDPs

Memory can be viewed as expanding the expressiveness of the policy class. **Memoryless policies form a strict subset of history-dependent policies**, and there exist POMDPs for which no memoryless policy can achieve optimal performance [28]. Introducing memory—whether finite or unbounded—enables the agent to compress relevant historical information into an internal state and thereby approximate optimal behaviour.

3.1 Memory mechanisms in reinforcement learning

Modern reinforcement learning employs a wide range of memory mechanisms that trade off expressiveness, computational cost, and ease of training. The most theoretically principled approach is to maintain an explicit information state using Bayesian filtering when a model of the environment is available. Approximate inference methods, such as particle filters or variational techniques, can be used to scale information state tracking to larger problems [40]. Still, these approaches remain computationally expensive and model-dependent.

A simpler and widely used heuristic is to provide the agent with a fixed window of recent observations and actions. This approach, often referred to as **finite history** or **frame stacking**, captures short-term temporal dependencies and is common in vision-based reinforcement learning. However, finite histories cannot represent dependencies that extend beyond the chosen window and therefore fail in tasks requiring long-term memory.

More structured finite-memory policies are provided by **finite-state controllers**, which represent policies as automata with a small number of internal states updated based on observations [23]. These controllers offer bounded complexity and interpretability but are challenging to scale to high-dimensional problems.

The dominant practical approach to memory in contemporary reinforcement learning is the use of **recurrent neural networks**, illustrated in Figure 3.1. Recurrent architectures maintain a hidden state that is updated at each time step as a function of the previous hidden state, the agent’s action, and the new observation:

$$h_{t+1} = f_{\theta}(h_t, a_t, o_{t+1}), \quad (3.1)$$

Architectures such as Long Short-Term Memory (LSTM, illustrated in Figure 3.2) networks [24] and Gated Recurrent Units (GRU) [14] mitigate vanishing gradient issues and enable learning long-range dependencies. Recurrent policies and value functions are strong baselines across a wide variety of POMDP benchmarks [37].

Beyond recurrence, more expressive memory architectures introduce explicit external memory structures. **Memory-augmented neural networks**, such as Neural Turing Machines and Differentiable Neural Computers, allow agents to store and retrieve structured information over long horizons [19]. More recently, **attention-based and transformer architectures** (illustrated in Figure 3.3) have been applied to reinforcement learning, enabling agents to selectively attend to past trajectory elements [51, 39]. While powerful, these models introduce additional computational challenges and lack strong inductive biases for online information state tracking.

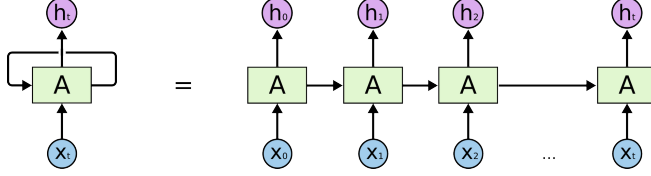


Figure 3.1: Unrolled RNN. [4]

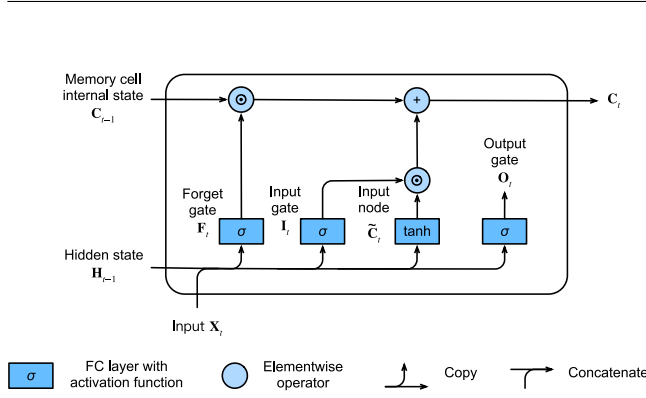


Figure 3.2: LSTM cell internal gates. [2]

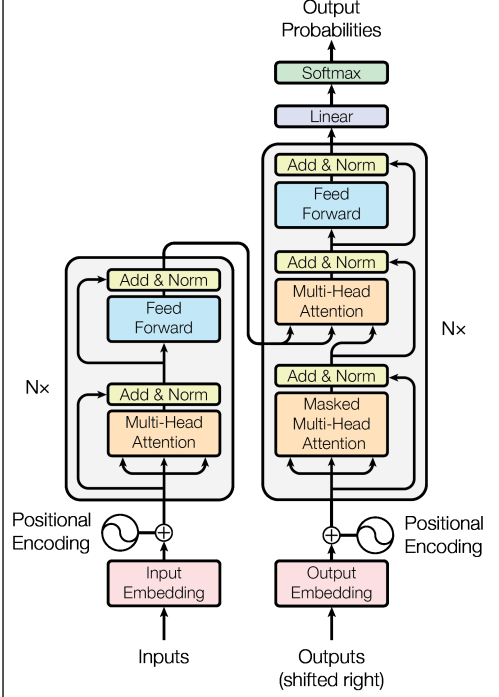


Figure 3.3: Transformer model. [51]

3.2 Explicit memory mechanisms

A large family of memory approaches in RL makes storage and retrieval **explicit** rather than implicit in a recurrent hidden state. These approaches share the goal of making important past information directly addressable at decision time, but they differ along several design axes (the structure of stored items, addressing schemes, update rules, and capacity management). Below are summarised standard explicit-memory designs and their practical trade-offs.

Differentiable external memories. Models such as **Neural Turing Machines** (illustrated in Figure 3.4) and **Differentiable Neural Computers** (illustrated in Figure 3.5) learn differentiable read/write operations to an external memory matrix, supporting flexible addressing and algorithmic-style manipulation of stored vectors [18, 19]. In RL, these controllers enable the storage of episodic facts or intermediate computations that can be retrieved many steps later, which is helpful for tasks that require explicit bookkeeping (e.g., counting, stack-like procedures, or remembering task-specific variables). In practice, end-to-end RL training of these controllers can be unstable; successful applications often rely on supervised pretraining, curriculum learning, or hybrid objectives to stabilise learning.

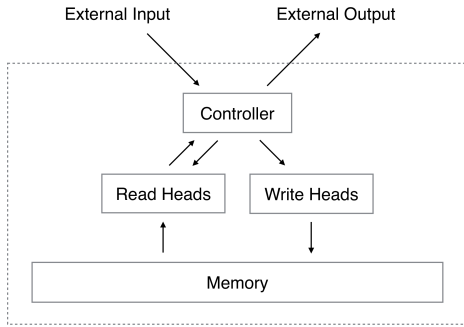


Figure 3.4: Neural turing machine.[18]

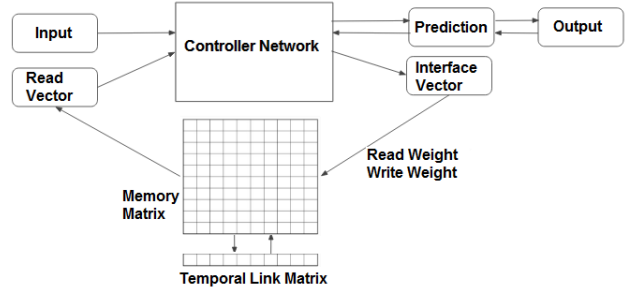


Figure 3.5: Differentiable neural computer. [7]

Structured spatial and map-like memories. For navigation and embodied agents, memories that mirror the environment geometry provide strong inductive biases. **Neural Map** style memories and learned spatial memory grids tie write/read operations to an egocentric or allocentric pose, making them naturally suited to localisation and mapping under occlusion [38]. These map-like representations excel when the task requires persistent knowledge of landmarks, object locations, or traversable areas, and they facilitate interpretable memory inspection (e.g., visualising map layers). Their limitations include the need for pose estimation or alignment and potential scale issues in large environments.

Episodic and non-parametric memory. Episodic modules store distinct trajectory snippets, key-value pairs, or entire episodes in a (often non-parametric) repository and perform nearest-neighbour or associative retrieval at decision time. Techniques such as episodic control and retrieval-augmented agents enable fast bootstrapping from previously successful trajectories and can markedly improve sample efficiency in sparse-reward settings [25, 52]. Non-parametric memories are attractive for rapid adaptation but require explicit capacity and eviction management to avoid unbounded growth and retrieval latency.

Slot-based and key-value memories. Slot-based designs allocate a fixed number of learnable slots or an indexable key space to compress and organise stored information (for example, object-centric slots for scene representations). Read/write is typically performed with attention or key-value lookups, combining compact parametric storage with flexible retrieval. Slot-based memories balance expressiveness and computational cost and are particularly useful when the number of persistent entities (objects, goals) is small relative to the episode length.

Compressed and hierarchical memories. To scale to long horizons, compressed-memory schemes summarise older events into denser representations that retain salient information while freeing capacity for recent observations (for example, compressive Transformers or hierarchical sketches) [42]. Hierarchical memories maintain fine-grained recent history alongside coarse, long-lived summaries (e.g., maps or policy-level facts), enabling selective retention of task-relevant information. These approaches trade off fidelity for capacity but allow attention-based or associative retrieval over much longer effective histories.

Practical concerns and design trade-offs. Explicit memory mechanisms introduce many design choices—**addressing** (content vs. location), **write policies** (learned vs. heuristic), **capacity/eviction policies**, and **retrieval latency**—that must be tuned per domain. They often improve interpretability (for example, readable memory slots or map layers) and enable targeted interventions (clearing or freezing particular memory regions), but they complicate training: replay buffers must preserve contiguous sequences; off-policy corrections become more intricate for memory-augmented agents; and distributed training can exacerbate staleness of stored memories [30]. For these reasons, careful engineering of memory interfaces and evaluation under realistic deployment constraints is crucial when adopting explicit memory modules.

3.3 Recent advances in memory mechanisms

Recent years have seen rapid progress in adapting sequence-modeling¹ and external-memory ideas to reinforcement learning. Advances cluster around three complementary themes: leveraging scalable sequence models, improving long-range retention through compression and retrieval, and constructing principled benchmarks and hybrid architectures that trade off expressiveness and deployability.

¹Sequence modelling is the task of learning from ordered data where each element depends on previous ones; in reinforcement learning, this typically means modelling trajectories of states, actions, and rewards.

Sequence modelling and Decision Transformer paradigms. Recasting policy learning as sequence modelling—where policies are implemented as causally-masked² Transformer decoders conditioned on past states, actions, and desired returns have become influential. Positional encodings (illustrated in Figure 3.6) provide the model with an explicit notion of order, while autoregressive masking and causal self-attention (illustrated in Figures 3.7 and 3.8) ensure that predictions depend only on past and present context. The **Decision Transformer** showed that attention-based sequence models can generate effective action sequences in offline RL³ by attending directly across trajectory context windows⁴, sidestepping some of the difficulties of temporal credit assignment via recurrence⁵ [10]. This line of work highlights the scalability and representational power of **self-attention** for memory-intensive tasks and has spawned many extensions that integrate conditioning signals, conserved representations, and imitation-like pretraining.

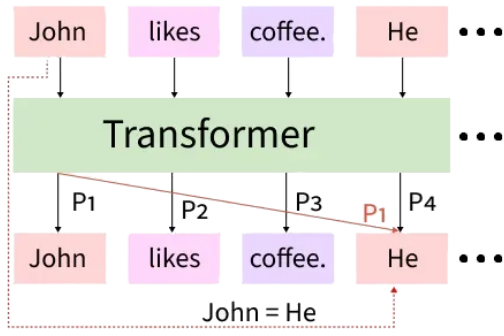


Figure 3.6: Positional encoding in a Transformer. [3]

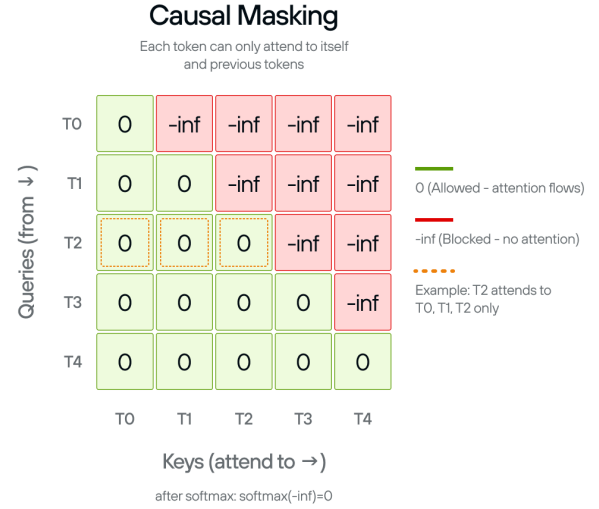


Figure 3.7: Causal attention mask. [1]

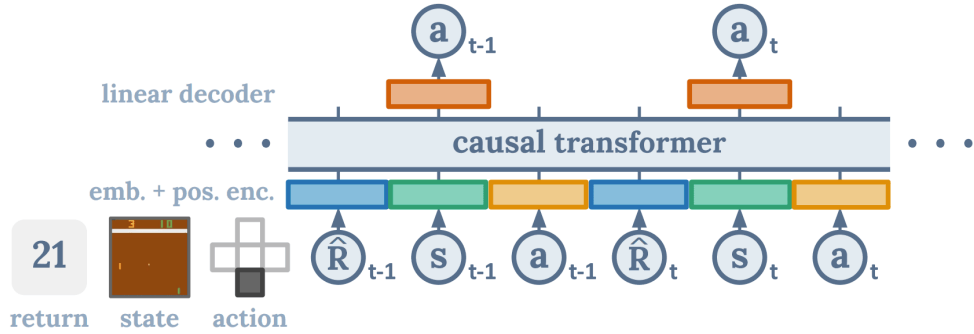


Figure 3.8: Causal Transformer. [35]

²Causal masking prevents a sequence model (such as a Transformer) from accessing future tokens or time steps during prediction, ensuring that outputs depend only on past and present information.

³Offline reinforcement learning refers to learning a policy from a fixed dataset of previously collected interactions, without further environment interaction during training.

⁴A trajectory context window is a finite segment of a trajectory (a sequence of states, actions, and possibly rewards) provided to a model as context for predicting or selecting the next action.

⁵Temporal credit assignment via recurrence refers to learning which past states or actions are responsible for current rewards by propagating learning signals backwards through time in a recurrent model.

Compressed-memory and retrieval-augmented designs. Long-range retention has been advanced by compressed-memory Transformer variants that summarise or compress older context into condensed representations that remain queryable (for example, compressive Transformers and memory-compressed attention) [42]. Complementary retrieval-augmented designs store salient past trajectory elements in an indexable repository and retrieve them via learned or approximate nearest-neighbour lookups at decision time; these hybrids combine the strengths of parametric sequence models with scalable non-parametric access and have shown benefits across diagnostic memory tasks as well as navigation and manipulation domains.

Benchmarks and principled evaluation protocols. A proliferation of benchmarks and evaluation protocols now aims to measure whether an algorithm truly benefits from memory. Suites such as **POBAX** and **MIKASA** formalise memory-improvability and provide controlled domains (localisation, visual control, manipulation), together with recommended protocols for reproducible evaluation under partial observability [50, 11]. Such benchmarks help disentangle genuine memory-dependent gains from incidental performance differences and encourage reporting of ablations⁶ (memory vs memoryless baselines, capacity sweeps⁷, and generalisation tests).

Hybrid architectures, efficiency, and distillation. Practical advances include hybrid models that combine recurrence with attention (for example, recurrently-computed keys or compressed recurrent Transformers), retrieval-augmented control that leverages episodic stores for sample efficiency, and distillation techniques that reduce inference cost of large Transformer-based learners for deployment on resource-constrained agents [6]. These hybrids aim to preserve long-range expressivity while controlling compute and memory footprints in online settings.

⁶Ablation studies systematically remove or modify components of a model to measure their individual contribution to overall performance.

⁷A capacity sweep evaluates performance while systematically varying a model’s capacity (for example, memory size, number of parameters, or context length).

3.4 Practical implications, applications, and outlook

Incorporating memory into reinforcement learning algorithms introduces several practical challenges. Memory-based agents require sequence-consistent training data: experience replay buffers must store and sample contiguous sequences rather than independent transitions, and recurrent hidden states must be handled carefully in off-policy learning [30]. Memory also interacts strongly with exploration, as agents may need to take information-gathering actions to reduce uncertainty before exploiting accumulated knowledge.

Memory capacity presents a fundamental trade-off between expressiveness and generalisation. Richer memory representations can capture complex dependencies but may increase sample complexity and overfitting. Recent work explores architectural biases that encourage fast learning and controlled forgetting to address this issue [36]. Partial observability further exacerbates the challenge of long-term credit assignment, since rewards may depend on events far in the past.

The importance of memory is evident across many application domains. In navigation and robotics, agents must remember landmarks and past observations to localise and plan under occlusion. In games and simulated environments, benchmarks such as T-Maze, RockSample, and flickering-observation tasks explicitly test memory requirements. Dialogue systems rely on memory to track conversational context, while healthcare and finance involve latent regimes and long-term temporal dependencies that cannot be inferred from instantaneous observations alone.

Taken together, developments in sequence modelling, compressed and external memory modules, and principled benchmarking have matured into practical tools for studying memory in POMDPs [10, 6, 50, 11]. Future progress will likely hinge on three complementary directions. First, improving **training stability** for memory-augmented and attention-based agents—for example, by adopting better optimisation practices, hybrid objectives, or more robust pretraining—will be necessary to make these models reliable in online and off-policy settings. Second, designing principled **eviction and compression schemes** that are aligned with downstream objectives (rather than heuristic capacity controls) will allow agents to retain task-relevant information over long horizons without unbounded resource growth. Third, developing **evaluation protocols** and benchmarks that more reliably expose when—and why—memory helps (including diagnostics for what is stored, when it is accessed, and how it affects decisions) will be essential to move beyond aggregate performance metrics and toward interpretable, deployable memory systems [50, 11, 6].

In summary, **memory is a fundamental requirement for reinforcement learning under partial observability**. While information state theory provides a formal foundation, practical agents rely on approximate and learned memory representations. Recurrent neural networks remain widely used, but external memory and attention-based architectures are increasingly explored. Addressing challenges in training stability, exploration, interpretability, and benchmarking is essential for advancing reinforcement learning methods that operate robustly in realistic, partially observable environments.

Chapter 4

State Representation Learning

State representation learning (SRL) studies how reinforcement learning (RL) agents can construct compact, informative internal representations of their interaction history to support effective decision-making. While classical RL theory assumes access to a fully observed Markovian state, real-world agents must typically infer it from high-dimensional, noisy, and often partially observable observations. As a result, representation learning is a central factor in sample efficiency, generalisation, stability, and scalability of modern RL systems.

Recent work emphasises that state representations are not merely intermediate features but a core design choice that mediates the interaction among perception, memory, prediction, and control. This chapter reviews the main principles and methods of state representation learning, with a particular focus on deep RL agents operating under partial observability. We follow the conceptual organisation proposed by recent surveys [16], covering classical foundations, neural approaches in model-free and model-based RL, representation learning with memory, evaluation methodologies, and emerging trends.

4.1 Motivation and problem setting

A suitable state representation should preserve information relevant for predicting future rewards and environment dynamics, while discarding nuisance variability unrelated to control. In fully observable, low-dimensional environments, handcrafted state variables or feature encodings may suffice. In contrast, high-dimensional sensory inputs such as images or proprioceptive streams often contain redundant, noisy, or aliased information, making manual feature design impractical.

From a theoretical standpoint, an optimal state representation corresponds to a sufficient statistic of the agent’s interaction history for optimal control. Under partial observability, this sufficient statistic is the information state over latent environment states. However, learning exact information states is generally intractable, motivating approximate representations optimised for surrogate objectives such as predictability¹, reward relevance², or controllability³ [16]. These objectives are closely coupled to the downstream learning algorithm⁴ and to the agent’s memory mechanism.

¹Predictability—how well the representation supports predicting future observations or latent variables.

²Reward relevance—the extent to which encoded features are informative about future rewards or value.

³Controllability—The degree to which the agent’s actions can influence encoded features.

⁴Downstream learning algorithm—The policy/value learner or planner that consumes the learned representation.

4.2 Classical approaches to state representation

Early reinforcement learning systems relied on manually designed feature mappings, including tile coding, radial basis functions, and linear combinations of basis functions⁵. Unsupervised dimensionality reduction techniques such as principal component analysis (PCA) or independent component analysis (ICA) were sometimes applied as preprocessing steps. While these approaches can reduce variance and computational cost, they are task-agnostic and ignore the sequential and control-dependent nature of RL.

Predictive State Representations (PSRs) provide an alternative classical framework in which the state is implicitly defined as a vector of predictions about future observable events, conditioned on action sequences. PSRs avoid explicit latent variables and offer strong theoretical guarantees, but their practical adoption has been limited by learning complexity and scalability issues.

These classical methods introduced key ideas (compression⁶, prediction, and sufficiency⁷) that continue to inform modern neural approaches, but they lack the flexibility required for large-scale, partially observable environments.

4.3 Neural state representation learning in model-free RL

In model-free⁸ deep reinforcement learning, state representations are most often learned implicitly as part of policy or value function approximation. Neural networks map raw observations (or observation histories) to latent features that parameterise policies, value functions, or both. While this end-to-end paradigm has achieved remarkable empirical success, it provides limited guarantees regarding representation stability, reusability, or interpretability.

To address these limitations, auxiliary learning signals are commonly introduced to shape representations explicitly [16]. Typical objectives include:

- **Reconstruction objectives**⁹, such as autoencoding observations or learned features.
- **Dynamics-based objectives**, including forward or inverse models that encourage temporal predictability.
- **Reward-based or value-based objectives**, which bias representations toward task-relevant features.
- **Contrastive and data-augmentation-based objectives**, which enforce invariances and robustness in latent space [33].

These auxiliary losses often improve sample efficiency and stability, particularly in partially observable settings. However, the survey literature emphasises that auxiliary objectives must be carefully aligned with control, as representations that are highly predictive or invariant may still be suboptimal for decision-making [16].

⁵Classic feature methods: tile coding uses overlapping binary tiles; RBFs use localised kernels; linear combinations form weighted sums of basis functions.

⁶Compression—reduce raw observations to a compact code that removes irrelevant variability.

⁷Sufficiency—retain the information necessary for optimal decision making so the representation acts as a sufficient statistic for control.

⁸Model-free: learns policies/values directly; Model-based: learns dynamics/reward models and uses them for planning.

⁹Train an encoder/decoder to reconstruct observations, encouraging descriptive, compressive embeddings.

4.4 Representation learning in model-based reinforcement learning

Model-based reinforcement learning explicitly learns a model of the environment’s dynamics, optionally including a reward model. In this context, state representation learning is typically framed as learning a latent dynamics model in which observations are encoded into a compact latent state, transitions are modelled in the latent space, and planning or policy optimisation is performed using the learned model.

Latent dynamics models must balance expressiveness with simplicity. Stochastic latent-variable models are particularly effective in partially observable environments, as they can model uncertainty about the hidden state. Learning objectives commonly combine reconstruction or likelihood terms with transition prediction losses, often using variational inference techniques. World model approaches¹⁰ exemplify this paradigm, demonstrating that planning in learned latent spaces can substantially improve data efficiency in high-dimensional domains [21].

The survey highlights that representation quality is a dominant factor in model-based performance: errors in latent dynamics can compound over long horizons, and choosing the representation and training objective are critical [16].

4.5 State representation learning under partial observability

Under partial observability, instantaneous observations are insufficient for optimal control, requiring representations that integrate information over time. A standard solution is to augment representation learning with memory mechanisms, such as recurrent neural networks, whose hidden states serve as learned summaries of past observations and actions.

Beyond recurrence, recent work explores representations that approximate information states without explicitly maintaining information state distributions. Predictive representations, contrastive sequence modelling, and self-supervised temporal objectives¹¹ have all been proposed as scalable alternatives [16].

Memory introduces additional challenges: representations must remain stable over time, selectively forget irrelevant information, and remain useful for control. Recent approaches emphasise structured memory and controlled forgetting mechanisms [36]. Transformer-based architectures provide an alternative by using attention over long histories, blurring the distinction between state representation and memory itself [39]. The interaction between representation learning and memory design remains an open research problem.

¹⁰World model approaches—learn a latent dynamics model (encoder, transition, decoder, reward) and plan or control in latent space.

¹¹These approaches learn temporal structure without external labels: predictive representations encode history to forecast future observations or rewards; contrastive sequence modelling uses contrastive losses on sequence segments to learn temporally discriminative embeddings; and self-supervised temporal objectives construct prediction or discrimination tasks (e.g., next-step prediction, ordering) from raw sequences to shape useful representations.

4.6 Evaluation of learned state representations

Evaluating state representations independently of downstream performance remains challenging. The literature identifies several complementary strategies:

- **Downstream RL performance**, measuring sample efficiency and asymptotic return.
- **Transfer and generalisation**, assessing whether representations learned on one task accelerate learning on related tasks.
- **Probing and diagnostic tasks**, using simple predictors to extract known latent factors from the learned representation.

Recent surveys caution that no single metric adequately captures representation quality, and that downstream performance alone can obscure essential failure modes¹² [16]. As a result, multi-faceted evaluation protocols¹³ are increasingly recommended.

4.7 State-of-the-art methods and current trends

Current trends in state representation learning emphasise scalability, robustness, and integration with planning and memory. Contrastive learning and augmentation-based objectives have become popular due to their empirical effectiveness and reduced reliance on reconstruction losses [33]. World models that combine latent dynamics with planning or policy learning continue to demonstrate strong performance across domains [21].

Transformer-based agents represent a shift toward treating the state as a dynamically constructed object derived from attention over experience, rather than as a fixed latent vector [39]. Another emerging direction is modular, structured representations, where separate components capture dynamics, rewards, uncertainty, or controllable factors, thereby improving interpretability and transfer [16].

Overall, state representation learning remains a foundational challenge in reinforcement learning. Progress increasingly depends on principled objective design, scalable architectures, and careful empirical evaluation, particularly in partially observable environments where representation and memory are tightly intertwined.

¹²Common failures include discarding task-relevant information, overfitting, learning control-irrelevant yet predictive features, and compounding model errors.

¹³Combine downstream performance, transfer/generalisation tests, probing tasks, and robustness/ablation checks.

Chapter 5

Environments for Reinforcement Learning with Memory

The previous chapters established that memory is essential for reinforcement learning under partial observability (Chapter 3) and that learned state representations serve as a key abstraction for storing and updating task-relevant information (Chapter 4). Building on that theoretical foundation, this chapter surveys the landscape of environments specifically designed to evaluate memory in RL agents.

Memory-oriented benchmarks test whether an agent can retain information over extended periods, integrate multiple past observations into a consistent internal representation, reason about temporal dependencies and hidden state variables, and use past experience to guide future decisions. Over the last decade, a variety of such environments have been proposed, differing in complexity, procedural variability, temporal structure, and the specific memory modalities they probe. Most existing environments, however, isolate a single memory demand at a time, which limits their ability to capture the multi-faceted memory requirements of realistic tasks.

This chapter provides a comprehensive survey of major RL environments that challenge an agent’s memory. It first defines the different forms of memory encountered in sequential decision making, referencing their theoretical underpinnings from earlier chapters. It then reviews classical and modern benchmarks, categorising them by domain type (simple puzzle, grid-world, 3D first-person, or roguelike) and analysing their mechanics, observability, procedural generation, and temporal dynamics. A comparative table and critical discussion of strengths and limitations follow. Finally, it identifies gaps in the current landscape—such as isolated memory tests, insufficient relational complexity, and limited multi-timescale structure—and uses these insights to motivate the design of a new, integrative memory benchmark introduced in the next chapter.

5.1 Memory Modalities in Benchmark Design

While Chapter 3 detailed why partial observability forces an agent to maintain an internal memory, this section operationalises that requirement into a taxonomy of memory modalities that benchmarks aim to test:

- **Working memory** – retaining recently perceived information for immediate decisions (e.g., the last few sensor readings). It is essential when the current observation alone is insufficient to infer the underlying state.
- **Sequential memory** – remembering ordered sequences of events, symbols, or instructions over longer horizons (e.g., a list of actions to be executed later).
- **Spatial memory** – constructing and recalling an internal map of an environment, as required in navigation tasks where previously seen corridors or rooms must be remembered.
- **Temporal memory** – tracking time intervals, periodic patterns, or the timing of events (e.g., a door that opens every 30 seconds).
- **Episodic memory** – recalling specific, temporally situated events from the current trajectory (e.g., “I saw the blue key in the third room I visited”) and later using that memory to guide behaviour. Episodic memory is about *what* happened *when* and *where*.
- **Relational memory** – learning and storing abstract associations or dependencies between entities (e.g., “blue keys open blue doors” or “pressing button A toggles gate B”) independent of the exact moment they were encountered. Relational memory captures *structural rules* that generalise across different episodes and contexts.

Real-world tasks rarely engage a single modality in isolation. A comprehensive benchmark should therefore integrate multiple memory challenges simultaneously. The following sections survey environments by the modalities they primarily employ, noting that most existing benchmarks focus on a narrow subset.

5.2 Classic and Simple Benchmarks

Early memory benchmarks were deliberately simple, designed to test whether a recurrent policy could preserve a small amount of information over a delay.

T-maze (Figure 5.2) is one of the most iconic experiments. In a typical setup, the agent observes a cue (e.g., a colour or symbol) at the base of a corridor. After traversing the corridor, it reaches a junction where the correct action depends on the remembered cue. The T-maze isolates delayed memory recall in a tightly controlled setting. It is computationally cheap, easy to analyse, and has been used extensively to compare recurrent network variants [29].

Despite its utility, the T-maze suffers from significant limitations:

- the environment is often deterministic;
- the layout is tiny and lacks exploration;
- memory demands are limited to a single binary cue;
- agents frequently overfit to narrow behavioural patterns.

Delayed Matching and Sequence Recall is another class of tasks that focuses on sequence retention: the agent must memorise a sequence of symbols, reproduce an action pattern after a distractor interval, or recall items from an earlier episode. These tasks directly probe working memory capacity and long-horizon credit assignment, but they typically abstract away navigation and physical interaction. Hence, they do not capture the spatial and relational reasoning required in more embodied RL scenarios.

MazeBase (Figure 5.1) [48] is a sandbox of simple 2D games designed to test reasoning and planning. It includes algorithmic tasks such as *if-then* logic, set operations, and small combat scenarios. The observation is usually fully observable (top-down grid of symbols), and actions are discrete. MazeBase enables curriculum learning and was used to benchmark abstract reasoning in deep RL. However, its visuals are rudimentary, it is no longer widely maintained, and it does not emphasise partial observability. It remains a historical point of reference for algorithmic memory tasks.

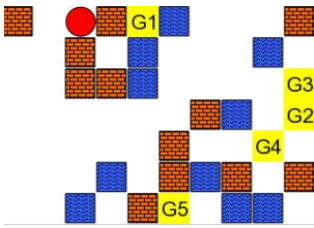


Figure 5.1: Example MazeBase task.

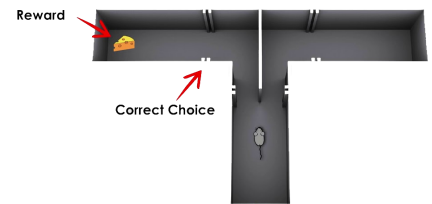


Figure 5.2: T-maze delayed cue task. [5]

5.3 Grid-World and 2D Environments

Grid-worlds strike a balance between simplicity and expressive task design. They are computationally efficient while supporting navigation, partial observability, and procedural generation.

MiniGrid (Figure 5.3) [13] is a lightweight, partially observable 2D grid-world framework. The agent typically sees only a small, egocentric window (e.g., a 7×7 tile view) and must accomplish tasks such as reaching a goal, picking up objects, unlocking doors with a key, or pushing blocks. Actions are discrete (move, turn, interact). Built-in tasks like **DoorKey**, **Memory**, and **KeyCorridor** explicitly require the agent to remember where a key was seen or which object it carries.

MiniGrid’s strengths include fast execution, a modular design, and easy customisation. Its simplicity, however, is also a weakness: many tasks are structurally simple, temporal dynamics are static, and object relationships rarely extend beyond single key–door pairs. While it is excellent for benchmarking core spatial and working memory under partial observability, it does not push the agent to integrate multiple memory types within a single episode.

BabyAI [12] extends MiniGrid by adding synthetic natural language instructions (e.g., “put the ball next to the red door”). The agent must parse the instruction and ground it in the grid-world. This introduces sequential and relational memory demands: the agent must retain parts of the instruction while navigating and interacting. BabyAI features a built-in expert curriculum and is often used to study sample efficiency in instruction-following. However, the language is restricted to a small grammar, and the grid-world complexity remains limited.

MiniWorld [13], a minimalistic 3D counterpart to MiniGrid, provides first-person visual navigation in small indoor environments. It supports procedurally generated rooms and is fast to simulate, but it lacks the rich relational mechanics (e.g., keys, doors, timers) that characterise memory-intensive tasks. It is primarily a lightweight platform for studying vision-based navigation.

MemoryGym (Figure 5.4) [41] is a suite explicitly designed to challenge memory-based agents. It comprises tasks such as *Mortar Mayhem*, *Mystery Path*, and *Searing Spotlights*. In *Mystery Path*, the agent must follow an invisible path indicated by memorised cues; in *Mortar Mayhem*, it must execute a randomly generated sequence of actions presented only at the start of each episode. The agent observes a partial top-down view of the environment, and each episode uses a fresh procedurally generated level. Temporal dynamics are tailored to each task (e.g., spotlights turning on and off). MemoryGym’s strengths lie in its clearly defined sequence-memory challenges and support for scalable difficulty through procedural generation. Its main limitation is that tasks tend to isolate a single memory modality (predominantly sequential memory) and use relatively simple graphics, which may allow agents to exploit task-specific shortcuts.

POPGym (Figure 5.5) [36] is a broad collection of over 15 POMDP tasks spanning games, puzzles, and control problems. Examples include *Battleship* (recalling hits on a hidden board), *Concentration* (pair matching), *Minesweeper*, *Memory Maze*, and *Pathfinder*. Observations are compact (small images or bitmaps), and actions are discrete or low-dimensional. All tasks are procedurally generated and offer multiple difficulty levels.

POPGym’s primary strength is its diversity: it collectively probes spatial, sequential, and strategic memory within a unified framework, with fast simulation. However, most tasks are designed to test a single cognitive ability in isolation. Embodied exploration is minimal, and long-term relational world structure is rare. POPGym is highly useful for controlled comparison of memory mechanisms, but does not offer integrated multi-modal challenges.

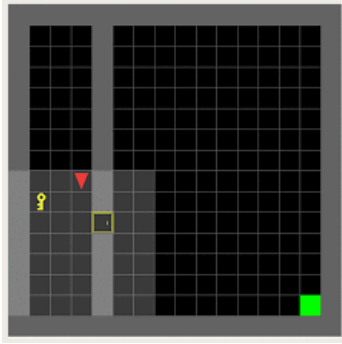


Figure 5.3: Minigrid

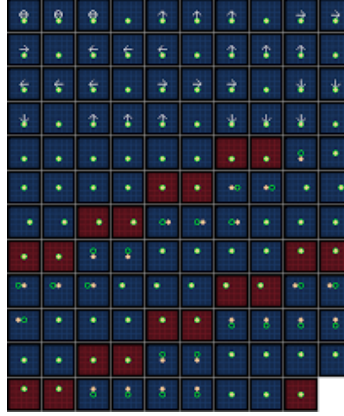


Figure 5.4: MemoryGym



Figure 5.5: POPGym

5.4 Complex 3D Environments

While grid-worlds allow rapid iteration, many researchers require richer sensory streams and more realistic physics to study the interplay of vision, control, and memory.

DeepMind Lab (Figure 5.8) [8] is a first-person 3D simulation built on the Quake III engine. The agent receives pixel observations (RGB or RGB-D) and can move, turn, strafe, and jump. DeepMind Lab offers a flexible level-design interface and supports procedural maze generation. Its built-in tasks include navigation mazes, laser-tag, and memory mazes, where the agent must explore a maze in darkness (revealing the map as it moves) and then navigate to a goal from memory.

DMLab-30 suite [17] extends DeepMind Lab with 30 diverse tasks, including language-conditioned navigation and “rooms keys light” (finding keys to illuminate rooms). DeepMind Lab excels at testing spatial and working memory under visual complexity, but the high computational cost of rendering 3D scenes slows down large-scale training. Relational and temporal reasoning beyond key–door interactions remains limited.

Psychlab (Figure 5.6) [34] is a set of cognitive psychology-style experiments implemented inside DeepMind Lab. Tasks include change detection, visual search, and multiple-object tracking. They provide controlled tests of visual working memory and attention, bridging RL and human cognition. However, Psychlab tasks are small-scale and often have known optimal strategies, making them less suitable for evaluating open-ended memory learning.

ViZDoom (Figure 5.9) [31] adapts the classic Doom game into an RL platform. The agent sees a first-person view of a 3D environment and can move, turn, and shoot. Scenarios range from basic combat to maze navigation with locked doors. **ViZDoom** is relatively lightweight, supports custom map generation, and has been widely used for vision-based RL. Memory challenges typically arise in navigation (e.g., finding a key in one room and remembering it while backtracking). However, most scenarios are combat-oriented and do not systematically isolate memory demands. Temporal aspects are simple, and relational mechanics are sparse.

Obstacle Tower (Figure 5.7) [27] is a procedurally generated 3D platformer built in Unity. The agent must climb the floors of a tower, each containing maze-like rooms with keys, doors, levers, and timed puzzles. Observations are RGB and depth; actions include movement, looking, and jumping. The robust procedural generation ensures that no two floors are identical, making memorising specific layouts impossible. Obstacle Tower explicitly tests spatial mapping, sequential puzzle solving, and temporal reasoning (e.g., rising flood levels). It is extremely challenging: even state-of-the-art agents struggle to progress beyond a few floors. Its main drawback is the high computational expense of rendering Unity scenes, which can slow research cycles.

ALFRED (Figure 5.10) [46] is a vision-and-language benchmark set in realistic 3D household environments (powered by AI2-THOR). The agent receives an egocentric RGB-D view and a natural language instruction (e.g., “put a glass of water on the table”). Tasks involve multi-step planning and interaction (navigate, pick up, open, pour, etc.). The partial observability and need to track subgoals while remembering the instructions make ALFRED a demanding test of sequential, relational, and spatial memory. Its difficulty and computational cost are very high, and baseline performance remains low. ALFRED highlights the gap between human instruction-following and current RL agents, but it is more a multi-modal comprehension benchmark than a pure memory suite.

NetHack Learning Environment (NLE) (Figure 5.11) [32] wraps the classic roguelike game NetHack. The agent views a partial 2D ASCII map (fog-of-war) and selects from a large discrete action set (move, use items, cast spells, etc.). The game is deeply stochastic, with rich item interactions and long episodes (many hundreds of steps). NLE naturally demands all memory modalities simultaneously: spatial mapping of the dungeon, sequential planning of item usage, episodic recall of quest items and traps, and relational bookkeeping of discovered properties. Its extreme procedural generation and difficulty make it a long-term research platform for exploration, planning, and memory. However, training is slow and high-variance, and only the best agents survive more than a few dungeon floors.

Crafter (Figure 5.12) [47] is a 2D open-world survival environment inspired by Minecraft. The agent sees a top-down pixel view with partial observability (only nearby areas visible) and has discrete movement and crafting actions. The world contains natural resources, monsters, and a crafting tree of 22 achievements (e.g., building a furnace, upgrading tools). Temporal dynamics include day–night cycles, regenerating resources, and periodic monster spawns. Crafter uniquely combines spatial mapping, multi-step planning (crafting dependencies), resource management, and survival under partial observability. Its strengths lie in integrating multiple memory demands within a single environment; its main weakness is the extreme difficulty posed by sparse rewards and a vast state space, which often requires reward shaping or simplified curriculum setups.

Proctgen (Figure 5.13) [15] provides 16 procedurally generated 2D arcade games (Coin-Run, Jumper, Maze, etc.) designed to benchmark generalisation. Observations are full-scene RGB images, and the games are essentially fully observable—the agent sees everything on the screen. Thus, Proctgen is not a POMDP benchmark per se, but it is mentioned here because it influences procedural generation. Memory demands are minimal; the challenge lies in visual generalisation and rapid control adaptation. Proctgen excels at testing out-of-distribution robustness, but it does not evaluate an agent’s ability to maintain internal state over time.

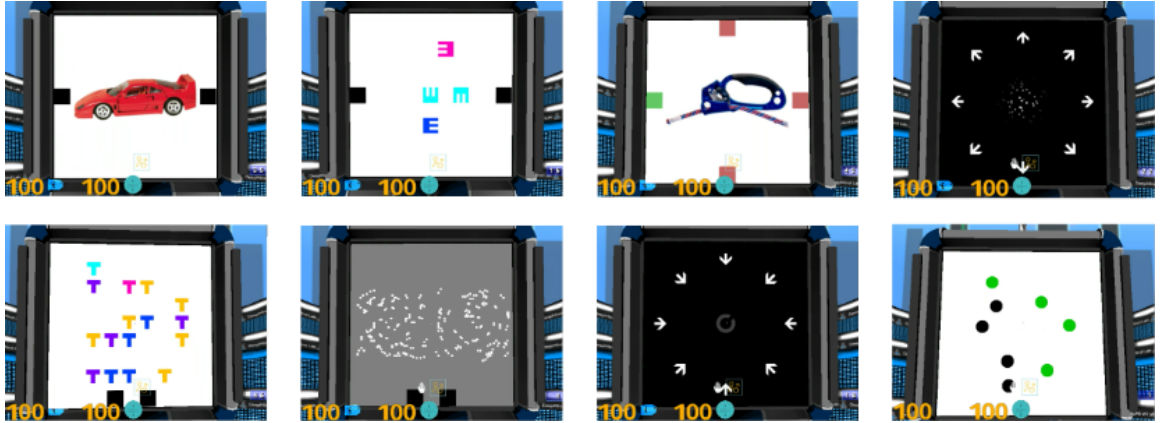


Figure 5.6: Psychlab – cognitive psychology experiments inside DeepMind Lab.



Figure 5.7: Obstacle Tower – procedurally generated 3D platformer.

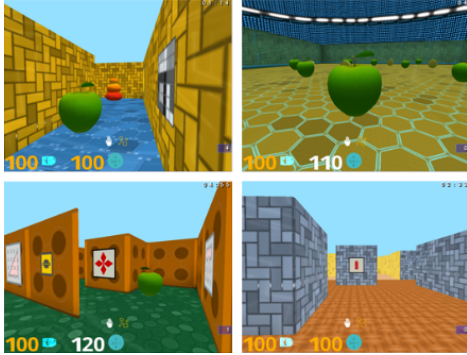


Figure 5.8: DeepMind Lab.

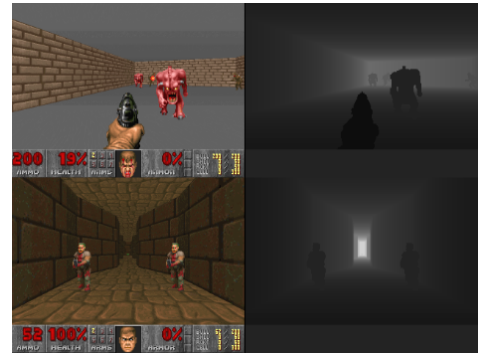


Figure 5.9: ViZDoom.



Figure 5.10: ALFRED.

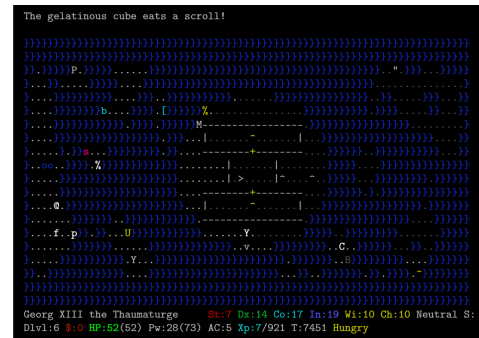


Figure 5.11: NetHack.



Figure 5.12: Crafter.



Figure 5.13: Procgén.

5.5 Comparative Analysis

Tables 5.1 and 5.2 summarise the key characteristics of the major memory-relevant environments discussed. It considers the domain type, partial observability, procedural generation (PCG), presence of relational mechanics (e.g., keys, doors, object dependencies), temporal dynamics, resource management, the primary memory modalities tested, and approximate computational cost of training (low, medium, high).

Table 5.1: Comparison of memory-focused RL environments (part 1)

Environment	Domain	Partial Obs.	PCG	Relational
MiniGrid/BabyAI	2D grid	Yes (local view)	Yes	Key-door, objects
MiniWorld	3D first-person	Yes	Yes	Low
MemoryGym	2D tasks	Yes	Yes	Minimal
POPGym	Mixed 2D	Yes	Yes	Varied (puzzle)
MazeBase	2D puzzles	No	Yes	Logical rules
DeepMind Lab/DMLab-30	3D first-person	Yes	Yes	Keys, doors, layouts
Psychlab	3D first-person	Yes	Limited	Low
VizDoom	3D FPS	Yes	Yes	Ammo, keys, health
Obstacle Tower	3D platformer	Yes	Yes	Keys, puzzles
NetHack (NLE)	2D roguelike	Yes	Yes	Items, quests, inventory
Crafter	2D survival	Yes	Yes	Crafting dependencies
Procgen	2D arcade	Yes (scrolling view)	Yes	Low
ALFRED	3D indoor	Yes	Limited	Object interactions

Table 5.2: Comparison of memory-focused RL environments (part 2)

Environment	Temporal	Resources	Memory Types	Cost
MiniGrid/BabyAI	Static	No	Spatial, Sequential, Working	Low
MiniWorld	Static	No	Spatial	Medium
MemoryGym	Episodic sequences	No	Sequential, Working	Low
POPGym	Mixed	No	Mixed (task-specific)	Low
MazeBase	Static	No	Algorithmic	Low
DeepMind Lab/DMLab-30	Moving elements, timers	Ammo, health	Spatial, Working	High
Psychlab	Rapid stimulus sequences	No	Visual working	High
VizDoom	Simple enemy movement	Ammo, health	Spatial	Medium
Obstacle Tower	Timed hazards	Keys	Spatial, Sequential	High
NetHack (NLE)	Long-term, turn-based	Food, health, tools	Episodic, Spatial, Relational	Medium
Crafter	Day-night, regeneration	Food, tools	Episodic, Planning, Relational	Medium
Progen	Moving platforms	No	Short-term	Low
ALFRED	Sequential steps	No	Sequential, Relational	High

The table reveals a clear trade-off between domain complexity and computational cost. Simple grid-worlds (MiniGrid, POPGym) allow rapid experimentation and sharp memory tests but lack relational richness and multi-timescale dynamics. High-fidelity environments (DeepMind Lab, Obstacle Tower, NLE) offer visual realism and integrated challenges, yet their training costs can be prohibitive for large-scale memory research. Few environments successfully combine relational reasoning, temporal dynamics, and resource management in a procedurally varied, partially observable setting. Only NetHack and Crafter come close, but they are extremely difficult for current algorithms and can be difficult to control for systematic memory ablation studies.

5.6 Limitations of Current Benchmarks

Across the surveyed environments, several recurring limitations emerge:

1. **Isolated memory modalities.** Most benchmarks test a single memory type at a time (e.g., remembering one cue, tracking one timer). Real-world tasks require the simultaneous integration of spatial, sequential, relational, and temporal memory.
2. **Limited relational complexity.** Object interactions are typically confined to simple key–door or button–switch rules. There is a lack of environments where distant regions are connected by non-trivial logical or topological dependencies (e.g., a button that opens a gate in a previously unseen area).
3. **Weak temporal dynamics.** Many environments are static or contain only short-lived periodic events. Agents are rarely forced to continuously update their internal beliefs about a dynamically changing world.
4. **Insufficient procedural diversity.** While procedural generation is common, layouts can be repetitive, allowing agents to overfit to structural motifs rather than develop general memory strategies.
5. **Absence of integrated long-term planning.** Resource management, survival constraints, and long-horizon trade-offs (e.g., exploring a risky area vs conserving energy) are seldom present. Episodes are often short, reducing the need for sustained memory.
6. **Lack of curriculum support.** Most benchmarks offer a fixed difficulty; there is no built-in mechanism to increase memory demands as the agent improves gradually.

These gaps suggest that the field might benefit from a new benchmark that deliberately combines multiple memory requirements into a single, procedurally generated, partially observable environment.

Chapter 6

Environment Design and Implementation

To overcome the shortcomings identified in Section 5.6, a memory-focused RL environment should integrate procedural topology with demands on spatial mapping, sequential recall, relational reasoning, and temporal coordination. It should operate across multiple timescales, impose resource constraints, support curriculum learning, and remain computationally efficient. This chapter introduces **GridMazeWorld** (Figure 6.3), a new environment that satisfies all of these criteria by combining procedurally generated, fully connected mazes with relational door–button mechanics, periodic timers, dynamic food sources, and a constrained energy budget, all under strict partial observability and with low computational cost.

6.1 Design Principles

- **Partial observability:** The agent sees only its 3×3 neighbourhood. This restriction forces it to build and maintain an internal representation of the world—a map, door states, and resource locations—across extended episodes.
- **Procedural variability:** Every episode constructs a new maze layout, button–door assignments, and food distribution. Without a static configuration, the agent cannot overfit to a single map; it must learn general memory and exploration strategies.
- **Relational dependencies:** Each button is linked to a particular door, and a door may open only when a working button on each of its sides is pressed. These non-local associations require the agent to remember which buttons correspond to which doors and whether they have been activated or broken.
- **Multi-timescale dynamics:** Doors operate on configurable periodic timers, button presses have immediate but time-limited effects, and food sources regenerate after variable delays. The agent must simultaneously track short-lived events and long-term resource cycles.
- **Resource constraints:** A constantly decaying energy budget and limited food supply create survival pressure. Efficient exploration and long-horizon planning are essential to gather food, activate buttons, and traverse doors before energy runs out.

6.2 Observation and action spaces

Internally, GridMazeWorld maintains a complete grid of $N = G \times G$ cells, each belonging to one of several tile types: empty, obstacle, food source, food item, door (closed or open), or button. In addition, the environment tracks a large amount of hidden information that is never directly revealed to the agent:

- **Button-door associations:** Every button is linked to exactly one door, but this mapping is not visible. The agent can only discover the connection by pressing a button and observing whether a door opens elsewhere.
- **Button state:** A button can be working or broken. A broken button no longer opens its door, but it visually looks identical to a working one. The only way to infer the state is to press it and remember the outcome.
- **Door timers:** Each door has an internal timer that counts down until it opens (automatic doors) or closes (all doors after being opened). The agent never sees these countdowns; it must infer the timing from repeated observations.
- **Food regrowth timers:** After a food source is harvested, it schedules a regrowth event after a variable delay. The agent is not told when a depleted source will regenerate.
- **Agent energy:** The exact numeric energy level is kept internally; the agent only receives a coarse, discretised indication.

All of these hidden variables turn the problem into a genuine partially observable Markov decision process (POMDP), where optimal behaviour requires the agent to build and maintain a memory of past events and inferred relationships.

Agent observation

At each time step, the agent receives a discrete observation vector of length 10:

Eight entries encode the eight cells immediately surrounding the agent (the 3×3 ring without the centre) as integer tokens: 0 for an empty cell (which may be a depleted food source that will later regrow), 1 for an obstacle, 2 for food source, 3 for food item (visible when food is present on a source), 4 for a closed door, 5 for an open door, and 6 for a button (which appears identical whether it is still functional or already broken).

The ninth entry encodes the last action taken by the agent (tokens 7–12).

The tenth entry encodes a discretised energy level, obtained by dividing the current energy into five equally spaced bins (tokens 14–18). The exact energy value is never shown.

The observation omits all temporal information: the agent does not see how long a door will stay open or closed, nor how many steps remain until a food source regrows. It also does not see the button-door mapping, and it cannot visually identify a broken button. These omissions force the agent to rely on memory for spatial relationships, temporal predictions, and past action outcomes.

Action space

The agent selects one of six discrete actions each step: move up, down, left, right, stay in place, or press a button. The button press action activates all button cells that are orthogonally adjacent to the agent; if any of them are working, the associated door opens, and the button may break with a fixed probability. The action space is therefore simple from a motor-control perspective, leaving the cognitive burden entirely on perception, memory, and planning.

6.3 Environment construction

Every episode starts by constructing a random, fully connected maze on a grid of N cells. The outer boundary of the grid is always filled with obstacles, forming an impassable wall. The remaining inner cells constitute the free space that will be populated with obstacles, doors, buttons, and food.

Obstacles must be placed so that the free space remains a single connected component (the agent can reach any empty cell without crossing a wall), because unreachable regions would waste exploration and violate the requirement that all goals (food, buttons) are attainable.

Food sources are the cells that periodically regenerate food items, which the agent can collect to replenish energy. Their spatial distribution strongly influences the difficulty of the episode: clustering food near the centre makes survival trivial, while pushing everything to the edges may force the agent to repeatedly cross dangerous regions. The placement algorithm should therefore produce a controlled mixture of central and spread-out sources, whose balance is tunable via the complexity level environment variable.

Doors act as navigational choke points, partitioning the maze into several disjoint regions when closed. Opening a door (by pressing its associated buttons or waiting for its periodic cycle) reconnects those regions, allowing the agent to pass. The placement of doors must ensure they are positioned at genuinely impactful locations, and the assignment of buttons must respect the map’s topology.

For every door that requires a **button** (about half of the doors, depending on the task class environment variable), the environment must distribute one button into each connected component that would be separated if the door were closed. Buttons may break with a configurable probability when pressed, introducing persistent consequences that the agent must track across the remainder of the episode.

6.4 Temporal dynamics and resource management

Once the static layout is generated, the environment simulates several dynamic processes every step:

- **Doors:** Automatic (non-button) doors toggle between open and closed after configurable timers. Button-controlled doors remain closed until a working button is pressed; after opening, they close again after the same `open_duration`. A door that has lost all its working buttons (because every button assigned to it broke) becomes permanently closed.
- **Food regrowth:** When the agent collects food from a source, the source becomes empty and schedules a regrowth event after a delay that increases with each consecutive collection. A priority queue maintains all pending regrowth events in order of their expiration time, so that it processes them in $O(\log F)$ per food collection (where $F \leq n_{\text{food}}$). Regrown food sources are ready for collection again instantly.
- **Energy:** The agent’s energy decays by a multiplicative factor and a small constant each step, and is replenished by a fixed amount when food is collected. If energy reaches zero or the maximum episode length is exceeded, the episode terminates.

These elements together create a multi-timescale environment: the agent must react to immediate energy loss, plan around the medium-term opening of doors, and anticipate the long-term regrowth of food.

6.5 API and configurability

The environment exposes a standard Gym-like interface with `reset`, `step`, and `render` methods, and provides a rich set of configuration parameters accessible through the constructor. In addition to the grid size and the maximum episode length, the user can control:

- obstacle density (as a function of a continuous `complexity_level`),
- number of doors and buttons per door (or let the environment compute them automatically based on grid size and `complexity_level`),
- door open/close durations and whether doors are periodic,
- button break probability,
- number and regrowth parameters of food sources,
- initial energy and per-step energy decay.

The environment also supports a `task_class` parameter that presets several of these values to produce four canonical problem types: `basic` (no doors or buttons, only food and obstacles), `doors` (automatic periodic doors), `buttons` (button-controlled doors), and `complex` (a mix of automatic and button doors). This design enables fine-grained curriculum learning and ablation studies.

6.6 Computational efficiency and used algorithms

A key design principle of GridMazeWorld is that it must run fast enough to allow hundreds of millions of environment interactions during deep reinforcement learning. All operations in a reset are linear in the total number of cells N . A single environment step performs a constant number of cell accesses, updates all doors ($O(D)$ with $D \ll N$), and processes food regrowth events from the priority queue ($O(\log F)$). As a result, GridMazeWorld can sustain step rates of several thousand per second on a single CPU core, making it suitable for large-scale experiments even with smaller Neural Network models.

Map and obstacle generation

A naive algorithm would repeatedly pick a random inner cell, turn it into an obstacle, and then run a flood-fill to verify that the remaining free cells still form a single connected component. If a disconnection is detected, the cell is restored. This process continues until the desired number of obstacles $n_{\text{obstacles}}$ has been placed. The flood-fill visits every cell in the worst case, taking $O(N)$ time per attempt. Because many random choices could disconnect the map, especially at high obstacle density, the total time is unpredictable.

Our implementation of the articulation point algorithm guarantees connectivity in a single pass, using a well-known graph-theoretic property: a vertex whose removal does not disconnect the graph is called a *non-articulation* point. If it only turns non-articulation points into obstacles, the remaining free space stays connected by definition. The algorithm proceeds as follows:

1. Start with a grid where all inner cells are empty, and the boundary cells are obstacles.
2. Treat the grid as an undirected graph whose vertices are all cells and whose edges exist between orthogonal neighbours that are both not obstacles.
3. Run Tarjan’s depth-first search (DFS) to compute, for each vertex, whether it is an articulation point. The DFS visits every vertex once and examines every edge twice, so the whole computation requires $O(N)$ time.
4. Collect all empty inner cells that are *not* articulation points. These are the cells that can safely be turned into obstacles without splitting the graph.
5. Shuffle the list of safe cells randomly, and mark the first $n_{\text{obstacles}}$ as obstacles.
6. As a final safety net, run a connected-component labelling pass over the free cells. If more than one component is found, carve straight passages from each smaller component to the largest, merging them into a single connected region. This step is also $O(N)$ and, in practice, is rarely triggered.

Because the initial empty grid (boundary walls only) is identical for every episode of a given grid size, the articulation point computation always yields the same set of safe cells. The algorithm therefore precomputes this set during environment construction and caches it. On each reset, steps 1–4 are replaced by a single array copy of the cached list, and only the shuffle, obstacle marking (step 5), and the safety labelling pass (step 6) are repeated. The resulting map is guaranteed to be connected, and thanks to the random shuffle, no two episodes produce the same layout.

Food source placement

A simple random placement. The most direct method would be to collect all empty inner cells into a list and then uniformly randomly select n_{food} of them to become food sources. While this yields an unpredictable distribution, it can easily create tight clusters or large empty areas, both of which reduce the pressure to explore and plan. It also offers no way to smoothly interpolate between a centre-biased and a uniform layout.

Centre-biased spread placement. This method achieves a more structured distribution in $O(N)$ time. It first collects all empty cells into an array and precomputes their Manhattan distances to the grid centre. The number of food sources n_{food} is either supplied directly or, when $n_{\text{food}} \leq 0$, drawn from a random range that shrinks as the complexity level increases: higher complexity yields fewer total food sources, increasing difficulty.

A fraction $(1 - \text{complexity_level})$ of the sources are placed near the centre. It builds a *centre pool* consisting of the n_{food} closest cells (plus a margin) and randomly chooses the required number from that pool. The remaining sources are placed according to a quasi-regular pattern: a spacing factor $k = \max(2, \lfloor \sqrt{N/n_{\text{food}}} \rfloor)$ is computed, and a random offset (o_y, o_x) is drawn. A candidate cell is accepted for a food source only if its coordinates satisfy $(y - o_y) \bmod k = 0$ and $(x - o_x) \bmod k = 0$. This spreads the sources roughly evenly across the map, with random jitter.

Because it only scans the list of empty cells once per phase and uses simple arithmetic, the entire placement runs in $O(N)$ time. The resulting arrangement guarantees that no large region is completely devoid of food while still allowing the centre to be slightly richer, a realistic property that encourages the agent to balance local foraging with long-range exploration.

Door placement

Template-guided placement (intuitive heuristic). A natural starting point is to identify cells where a door would act as a bottleneck. One can create a small library of 3×3 templates that encode local corridor or junction geometries (e.g., a vertical corridor with walls on the left and right, a T-junction, etc.), as shown in Figure 6.1. For every empty cell, it checks whether its local neighbourhood matches any template; only matching cells are considered door candidates. Additional global checks (e.g., flood-fill to verify that closing the cell indeed separates the map into at least two components) can then filter these candidates. This approach is intuitive and easy to illustrate. However, it is inherently limited: predefined templates may miss valid articulation points in irregular topologies, thereby placing doors at irrelevant locations, and flood-fill verification can become expensive if many candidates are tested.

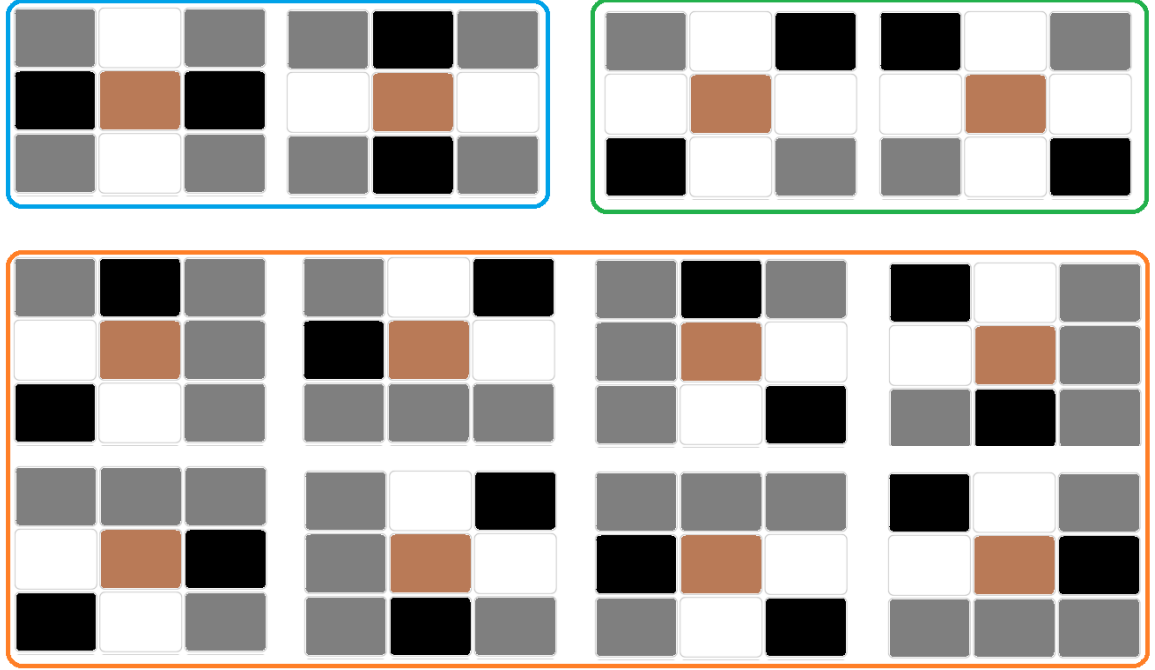


Figure 6.1: A set of 3×3 templates that describe local corridor and junction patterns. Cells that match such patterns are plausible door locations. (black: obstacle / white: passable / grey: do not care / brown centre: potential door placement)

articulation point based placement. GridMazeWorld instead recomputes articulation points on the *final* static grid (after obstacles and food sources are placed) and uses them as the sole source of door candidates. A cell is eligible to host a door if:

- it is an articulation point,
- its removal would split the graph into at least two connected components (the articulation point count is at least 2),
- each resulting component contains at least a minimum number of empty cells (to avoid placing doors on tiny dead-ends).

This set of candidates is then shuffled, and doors are picked sequentially, with a minimum pairwise Manhattan distance enforced to prevent doors from being too close together.

After the static layout (obstacles, food sources) is finalised, it runs articulation point computation on the resulting graph. This is an $O(N)$ pass that identifies every cell whose removal would disconnect the free space. From these articulation points, it selects door candidates, ensuring that every structural bottleneck in the maze is considered, regardless of its local shape. The candidate set is thus both principled and robust.

Button placement

For every door that requires a button (about half of the doors, depending on the task class environment variable), the environment must distribute one button into each connected component that would be separated if the door were closed. Buttons may break with a configurable probability when pressed, introducing persistent consequences that the agent must track across the remainder of the episode.

Naive flood-fill per door. A straightforward approach to button placement does not rely on the door cell’s articulation property. The door is temporarily treated as an obstacle, and a single flood-fill is launched simultaneously from its four orthogonal neighbours. As the wavefront expands, each visited cell is tagged with the neighbour from which it was first reached; if two wavefronts meet, their labels are merged (Figure 6.2), reflecting that those sides are actually connected through the maze. Once the entire reachable area has been scanned, the number of distinct remaining labels determines the number of separated components. If at least two components exist, one button is placed in each by randomly selecting an empty cell whose Manhattan distance to the door does not exceed the door’s open duration.

Although conceptually simple, this method scans every passable cell for every door, leading to a total cost of $O(D \cdot N)$ for D doors. For large mazes or many doors, the per-episode reset time quickly becomes prohibitive, motivating a more efficient alternative.

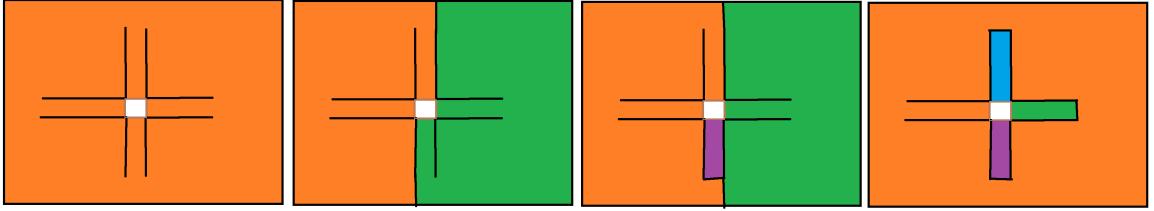


Figure 6.2: Illustration of connected components found by a full-grid flood-fill. Wavefronts (coloured) expand from the door’s neighbours; if they meet, their labels are merged.

Bounded BFS. The environment exploits the fact that every door is now placed on a verified articulation point. Because it already knows that removing the door cell disconnects the graph, a full-grid scan is unnecessary. Instead, a single BFS is started from the door’s four neighbours and restricted to a small radius—the door’s open duration minus a safety margin. Since the door is a true articulation point, the four search frontiers are topologically separated and can never meet; no component merging is required.

6.7 Conclusion

GridMazeWorld combines procedurally generated connectivity-preserving mazes, articulation-point-based door placement, region-based button assignment, and configurable temporal and resource dynamics into a single partially observable environment. Each component of the generation pipeline was carefully designed to be both theoretically principled and computationally efficient, while producing the multi-faceted memory demands (spatial, sequential, relational, and temporal) that the previous chapter identified as missing from existing benchmarks. The GridMazeWorld is a solid testbed for developing and comparing agents that must learn to maintain and use rich internal state representations.

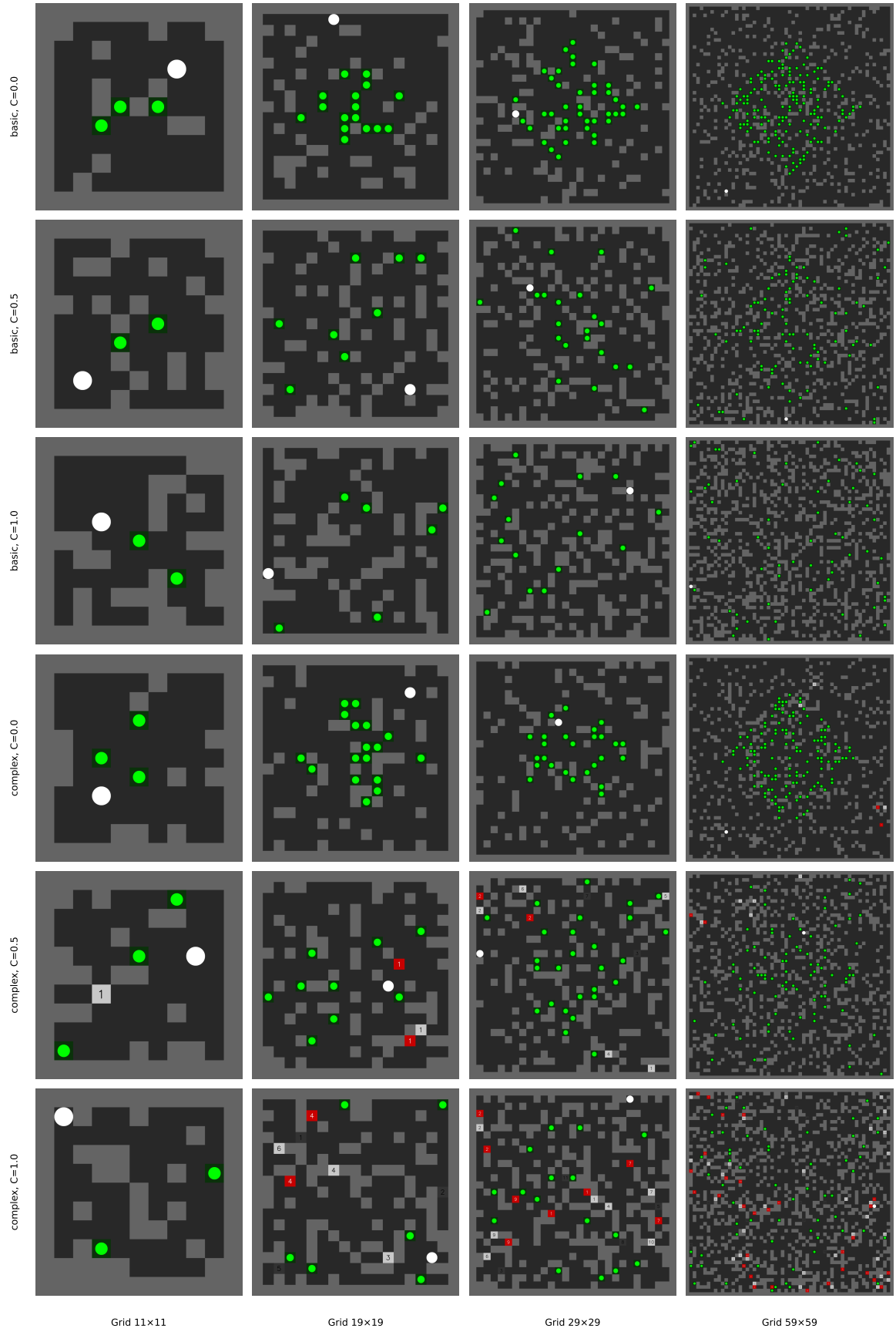


Figure 6.3: GridWorld Environments of various Grid Sizes, Tasks, and Complexity levels. (black: empty cell / grey: obstacle / green: food / white: closed door / red: button)

Chapter 7

Agent Architecture and Training Algorithms

The previous chapter described the GridMazeWorld environment, a partially observable domain that demands sophisticated memory and planning. This chapter details the agent architectures and training methods I used to tackle it. This chapter first presents the neural network families (LSTM and Transformer) and the auxiliary prediction tasks that shape their latent representations. Next, it describes the two policy optimisation algorithms used in the experiments: REINFORCE with baseline and Proximal Policy Optimisation (PPO [44]). Then it introduces the dynamic complexity manager, which implements an automatic curriculum that adapts the environment’s difficulty to the agent’s performance. Finally, it outlines the system’s data flow, the vectorised environment implementation, and the benchmarking tools that were essential for efficient experimentation.

7.1 Agent network architectures

All agents share a common interface: they receive a sequence of discrete observations of length 10 (the 8 neighbour tokens, last action token, and energy token), and output a categorical distribution over the 6 actions and, optionally, a value estimate (for PPO) and auxiliary predictions (for self-supervised state representation learning). The internal architecture determines how information is integrated across time steps and how the agent maintains a memory of past events.

LSTM model is illustrated in Figure 7.2. Each of the 10 observation tokens is embedded into a 512-dimensional vector, and a learnable positional encoding is added to preserve the order of tokens within an observation. The 10 vectors are flattened and passed through a three-layer MLP (with ReLU activations) that aggregates them into a single 512-dimensional hidden vector per time step. This aggregated vector is fed into a single-layer LSTM (hidden size 512).

The LSTM’s recurrent state is initialised to zero at the start of each episode and is carried forward across steps, providing the agent with an unbounded memory of the episode history. From the LSTM’s output hidden state, a linear layer produces the policy logits (6 actions). If the value head is enabled, another small MLP (hidden size 256) estimates the state value. When auxiliary tasks are used, two additional MLPs predict the next scalar energy and the next 10-token observation from the same LSTM state.

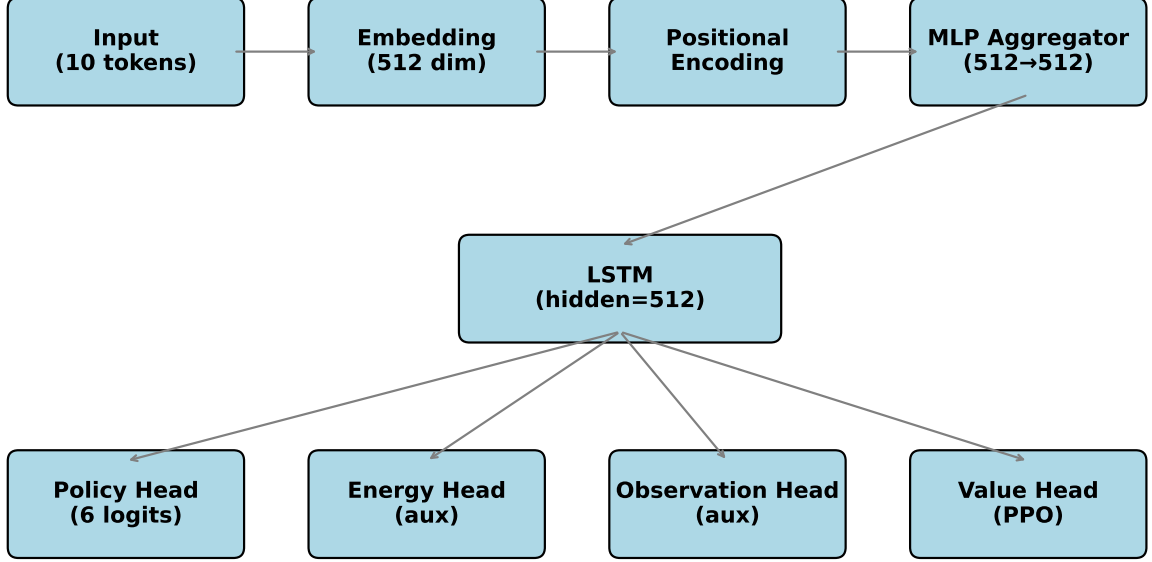


Figure 7.1: LSTM architecture

Transformer model (Figure 7.2) replaces the recurrent LSTM with a causal Transformer encoder, which can capture long-range dependencies without vanishing gradients, provided a sufficiently large context window.

As in the LSTM model, each observation’s 10 tokens are embedded and then augmented with a per-token positional encoding. Instead of an MLP aggregator, a scaled dot-product attention aggregator compresses the 10 token embeddings into a single 512-dimensional vector per time step. A sinusoidal temporal positional encoding is added to the entire sequence of aggregated vectors, giving the Transformer awareness of observation order.

The model prepends a set of learnable memory tokens (10 by default) to the sequence. These tokens are updated at each forward pass, and their final states are carried over to the next call, effectively extending the context beyond the fixed window length. The concatenated sequence (memory tokens + observations) is processed by a stack of three Transformer encoder layers (8 attention heads each, feed-forward dimension 2048) with a causal mask to prevent looking into the future.

The outputs corresponding to the observation positions are fed to the policy head (LayerNorm → linear → GELU → dropout → linear) to produce logits, and optionally to the value head (a two-layer MLP with ReLU). Auxiliary heads are attached to the same observation outputs.

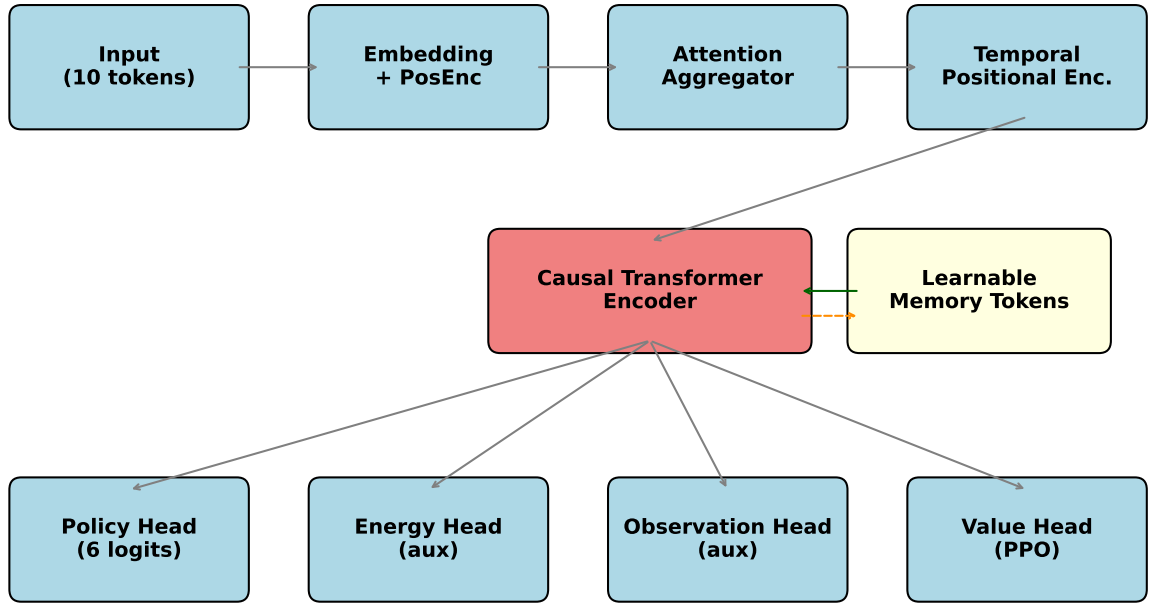


Figure 7.2: Transformer architecture

Auxiliary tasks

Auxiliary tasks are self-supervised prediction objectives that do not depend on the reward signal. They encourage the agent’s latent representation to capture environment dynamics, which is especially valuable in sparse-reward settings such as late-stage, complex mazes. Two auxiliary tasks are used:

- **Energy prediction:** from the current hidden state, the model predicts the scalar energy value that will be observed at the next step. The loss is the mean squared error.
- **Observation prediction:** the model predicts the next 10-token observation vector (treated as a regression problem). The loss is the cross-entropy.

Both auxiliary losses are added to the main RL loss with small coefficients (0.1 for energy, 0.05 for observation). Because the auxiliary tasks are only available during training, they are disabled during testing and evaluation.

7.2 Training algorithms

I implemented two policy optimisation algorithms: REINFORCE with baseline (a basic policy gradient method) and Proximal Policy Optimisation (PPO), which is more sample-efficient and robust.

REINFORCE with baseline updates the policy after each episode using the discounted return $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$. To reduce variance, it subtracts a baseline $b(s_t)$, effectively normalising the advantages to zero mean and unit variance. The policy loss is:

$$\mathcal{L}_{\text{PG}} = -\frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T \log \pi_{\theta}(a_t|o_t) \hat{A}_t, \quad (7.1)$$

where $\hat{A}_t$ are the normalised advantages. An entropy bonus $\mathcal{H}(\pi_{\theta}(\cdot|o_t))$ is added with coefficient 0.01 to encourage exploration. The loss is computed over a batch of parallel episodes (refer to section 7.4) and back-propagated through the full episode sequence without truncation (the whole episode is treated as one sequence). REINFORCE updates can be performed after every episode or after a full epoch of several parallel episodes.

Similar to **GRPO** (Group Relative Policy Optimisation) [45], this REINFORCE variant uses batch-level normalised advantages instead of a learned value function. However, unlike GRPO, it applies the standard policy-gradient objective directly on log-probabilities without PPO-style clipping, KL regularisation, or multiple optimisation steps over the same batch.

Proximal Policy Optimisation (PPO) uses a clipped surrogate objective [26] that restricts the policy change per update, leading to more stable training. For a batch of experience collected with the old policy $\pi_{\theta_{\text{old}}}$, the policy loss is:

$$\mathcal{L}_{\text{CLIP}}(\theta) = -\mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (7.2)$$

where $\rho_t(\theta) = \frac{\pi_{\theta}(a_t|o_t)}{\pi_{\theta_{\text{old}}}(a_t|o_t)}$. The advantage estimate $\hat{A}_t$ is computed via Generalised Advantage Estimation (GAE [43]):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(o_{t+1}) - V(o_t), \quad (7.3)$$

with the parameters γ (discount factor) and λ controlling the bias-variance trade-off. The value function is updated with MSE loss between the predicted value $V_{\theta}(o_t)$ and the target return $R_t = \hat{A}_t + V_{\theta_{\text{old}}}(o_t)$. The total loss combines the clipped policy loss, the value loss, and an entropy bonus:

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}_{\text{CLIP}} + c_1 \mathcal{L}_{\text{VF}} - c_2 \mathcal{H}(\pi_{\theta}), \quad (7.4)$$

where c_1 and c_2 weight the value function loss and the entropy regularisation term, respectively. PPO performs multiple epochs of mini-batch updates on the same collected rollout, using a vectorised environment to collect many trajectories simultaneously.

7.3 Dynamic complexity and curriculum learning

The environment’s difficulty is controlled by a continuous `complexity_level` (0–1) and a discrete `task_class` (`basic`, `doors`, `buttons`, `complex`). The curriculum manager, implemented in `dynamic_complexity.py`, adapts these parameters online based on the agent’s recent performance. Importantly, each task class maintains its own complexity value, so progress in one stage does not reset when the curriculum revisits it later.

Performance metric

At the end of every epoch, the manager computes a performance score. Let $\bar{r}$ be the average reward over the most recent `performance_window` episodes (a rolling window of raw rewards). The stage-dependent maximum reward r_{stage}^* is the highest average reward ever observed in the current stage (initialised to the first received $\bar{r}$). The performance score is then

$$s = \begin{cases} 0, & \text{if } r_{\text{stage}}^* < 0.1 \\ \min\left(1, \frac{\bar{r}}{r_{\text{stage}}^*}\right), & \text{otherwise.} \end{cases} \quad (7.5)$$

Thus, the score measures how close the agent is to its personal best in the current stage, and it is always clamped to $[0, 1]$.

Complexity adjustment

Every `adjustment_interval` epochs (default 50), the manager first checks whether at least half of the `performance_window` is full. If so, it calculates the performance score s and compares it against two thresholds:

- If $s > \text{increase_threshold}$ (default 0.6), the complexity of the current stage is increased by `step` = 0.05, up to `max_complexity` = 1.0.
- If $s < \text{decrease_threshold}$ (default 0.35), the complexity is decreased by the same step, down to `min_complexity` = 0.0.
- Otherwise, complexity stays unchanged.

When a change occurs, the rolling reward window is cleared to give the new difficulty a fair evaluation and the environment’s internal parameters are updated.

Stage switching

The curriculum decides when to move the agent to a different task class. Stage switching is suppressed if the curriculum contains only one stage, or if the agent is still in the **basic** stage and its complexity has not yet reached **min_basic_complexity** (default 0.2). Otherwise, a switch is triggered when any of the following conditions hold:

1. The number of epochs since the last complexity increase exceeds **stagnation_switch_interval** (default 250 epochs).
2. The standard deviation of the rewards in the rolling window is below 0.1 and the mean of those rewards is below the **decrease_threshold**.

During the first full cycle, the progression is linear: **basic** $\rightarrow$ **doors** $\rightarrow$ **buttons** $\rightarrow$ **complex**. When the agent reaches **complex** and completes it, the cycle restarts, but from that point onward, stage selection becomes probabilistic. For each subsequent switch, the probability of choosing a stage k is proportional to

$$p_k \propto (1 - c_k + \epsilon) \cdot (1 - f_k + \epsilon), \quad (7.6)$$

where c_k is the current complexity of stage k , f_k is the frequency with which the stage has been visited relative to the total number of switches (plus one to avoid zero), and $\epsilon = 0.1$ is a small constant that ensures no stage ever receives a zero weight. This mechanism favours stages with lower current difficulty and those that have been practised less often, thereby creating an automated, balanced curriculum.

Whenever a stage switch occurs, the rolling performance window is cleared, the “last complexity increase” timer is reset, and the stage-specific best reward r_{stage}^* is re-initialised to $-\infty$. This design allows the agent to first master navigation and survival, then gradually encounter doors, buttons, and mixed challenges without any manual scheduling.

7.4 System implementation and data flow

To achieve the throughput required for deep RL on large mazes, the environment was implemented in C++ and exposed to Python via PyBind11. The Python training code uses a **VectorizedMazeEnv** class that holds an array of **GridMazeWorld** instances and steps them all in parallel using OpenMP. The typical batch size is 64. The data flow for a PPO update is as follows:

1. The vectorised environment resets all instances, each with its own random seed. Observations are returned as a NumPy array, which is immediately copied into a pre-allocated GPU tensor **self.obs_tensor** of shape $[B, 1, 10]$ ($B = 64$).
2. At each environment step, the agent network performs a forward pass on **self.obs_tensor** (under **torch.no_grad()** to save memory during rollout) to produce logits, a value estimate, and auxiliary prediction if enabled. Actions are sampled from the categorical distribution and placed in a CPU buffer.
3. The **vector_env.step()** call passes the action array to C++ and executes one step in every environment instance concurrently. It returns updated observations, scalar rewards, termination/truncation flags, and a dictionary of metadata (e.g., current energy) for each environment.

4. All returned data is copied into pre-allocated tensors (`self.reward_tensor`, `self.done_tensor`) and appended to the rollout storage. If auxiliary tasks are active, the current energy and the next observation are also stored to serve as regression targets.
5. The loop continues until all environments have terminated or the maximum episode length is reached. The resulting trajectory (observations, actions, rewards, values, logits, and optional targets) is stacked into tensors of shape $[B, T, \dots]$. Where T is the number of steps.
6. After the episode ends, GAE is applied to compute advantages and returns. The rollout buffer is then used for several epochs of mini-batch updates according to the PPO clipped objective (detailed in section 7.2).
7. Throughout training, the environment exposes its internal grid, door states, and food positions solely for visualisation and debugging – these are never provided to the agent, preserving partial observability.

Pre-allocating the observation, reward, and done tensors, together with a dedicated CPU action buffer, avoids repetitive GPU allocations and host-device transfers during the rollout loop. The entire forward-pass and environment-step cycle runs synchronously, ensuring that the `step()` function is called with the actions generated by the latest policy.

Benchmarking and other tools

To ensure the system’s efficiency, I developed several tools (located in the `tools/` directory):

- `benchmark_cpp_env.py` – measures reset, and step latency of a single environment.
- `benchmark_vectorized_env.py` – vectorised version with batch size 64.
- `multiple_envs_screenshot.py` – used for generating Figure 6.3.
- `replot_experiments.py` – regenerates all training plots from saved metrics without re-running experiments.
- `run_experiments.py` – runs hyperparameter sweep experiments in section 8.1.

These tools were instrumental in identifying performance bottlenecks and guiding the optimisations described in the section 6.6.

The vectorised environment achieves a throughput of roughly 200 epochs per second for episodes of 100 steps with a batch size of 64 while generating complex task grids with a complexity level of 1.0 and a grid size of 19 on an Intel Core i5 8500. Benchmark timings:

Operation	Mean $\pm$ Std. dev.	Total time	Iterations
reset	$1104.35 \pm 334.60 \mu\text{s}$	0.110 s	100 vector resets
step	$35.84 \pm 6.94 \mu\text{s}$	0.358 s	10000 vector steps

The low per-step latency confirms that the C++ backend successfully removes environment interaction as a bottleneck during training.

Chapter 8

Methods and Experimental Protocol

This chapter describes the experimental setup used to evaluate the memory architectures and training protocols. All experiments were conducted using the GridMazeWorld environment described in Chapter 6 and the agent architectures described in Chapter 7. The primary training algorithm was PPO, with REINFORCE used for comparison. The hyperparameters were tuned on a basic task (grid_size = 11, complexity_level = 0.5, max_steps = 100) before scaling to larger grids and dynamic curricula.

The environment parameters for different experiment families are summarised in Table 8.1. Default parameters (food energy, initial energy, energy decay, door timers) were kept constant across all runs.

Table 8.1: Environment configurations for the main experiment families. “Dynamic” indicates the parameter changes during training.

Experiment family	Task class	Complexity	Grid size	Max steps
Hyperparameter tuning	basic	fixed 0.5	11	100
Grid size scaling	basic	dynamic	19	200
Network scaling	basic	dynamic	19	200
Task curriculum	all	dynamic per stage	19	200

8.1 Training algorithm and hyperparameter sweeps

All hyperparameter sweeps were performed on a fixed environment configuration: grid size 11×11 , task class **basic**, training complexity 0.5, maximum steps per episode 100, and no explicit food source count (randomised from complexity). Unless otherwise stated, the following base setup was used:

- Network: LSTM with hidden size 512, no auxiliary tasks.
- Algorithm: PPO with one intra-epoch, mini-batch size 64, batch size 64 parallel environments.
- Learning rate: 5×10^{-4} , optimiser: Adam.
- Training epochs: 5000 (adjusted in the batch-size sweep to keep total environment steps constant).
- Fixed PPO hyperparameters: discount $\gamma = 0.97$, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, value coefficient 0.5, entropy coefficient 0.01, gradient clipping norm 1.0.

Evaluation during sweeps. Every 100 epochs, the agent was evaluated on the same task class (**basic**) over a range of complexity levels centred at the current training complexity (± 0.2 , sampled every 0.05). For each complexity point, 2 epochs of 64 parallel environments were run with a stochastic policy, and the mean reward per environment was recorded. From these, per-complexity means the **median** was extracted as the primary score, accompanied by the first and third quartiles and the full range to summarise generalisation. In addition, two extra evaluations were performed *only at the training complexity* (0.5): one with a deterministic (argmax) policy and one with the stochastic policy, yielding point estimates of current performance. Within each sweep, the hyperparameter configuration that achieved the **highest median** test reward was chosen.

Each sweep varied a single hyperparameter while all others remained at the baseline values indicated above. The specific sweeps were:

1. **Algorithm comparison.** PPO (one intra-epoch) vs REINFORCE. For this comparison, PPO used a mini-batch size of 6400 and a single PPO update per collected batch.
2. **PPO intra-epochs.** After selecting PPO, I tested intra-epoch values 1, 2, 4 while keeping the mini-batch size at 6400. A single intra-epoch gave the most stable learning and was used as the baseline for all subsequent sweeps and larger experiments.
3. **Mini-batch size.** Using the chosen intra-epoch setting (1, to isolate the effect), I tested mini-batch sizes 64, 256, 1600. The smallest value (64) gave the highest median test reward and was retained.
4. **Batch size (parallel environments).** In order to maintain a constant total number of environment interactions, the number of training epochs was scaled inversely with batch size: $32 \times 10\,000$, 128×2500 . The original batch size of 64 had the highest median and was kept for subsequent experiments.
5. **Optimiser.** Compared Adam (default), AdamW, SGD with momentum 0.9, and RMSprop. Adam remained the default.
6. **Learning rate.** A sweep over 1×10^{-4} to 1×10^{-3} in steps of 1×10^{-4} was performed. The value 6×10^{-4} gave the highest median test reward and was adopted for all later experiments.

The final hyperparameter set carried forward to the large-scale experiments was: PPO with 1 intra-epoch, mini-batch size 64, parallel batch size 64, Adam optimiser, and learning rate 6×10^{-4} .

8.2 Dynamic complexity parameters

For experiments using the dynamic complexity manager, the following settings were used: performance window = 10, increase threshold = 0.60, decrease threshold = 0.35, complexity step = 0.05, adjustment interval = 50 epochs, stagnation switch interval = 250 epochs, minimum complexity for the basic stage = 0.2. The performance metric was computed as mentioned in section 7.3.

For dynamic curricula, the primary evaluation criterion was the highest complexity level the agent could sustain without regressing, as the median test reward was unreliable when difficulty was changing during training.

8.3 Reproducibility

To ensure reproducibility, a fixed random seed (42) was used for Python, NumPy, and PyTorch random number generators. Each environment instance in the vectorised wrapper was initialised with a deterministic seed derived from a base seed and an environment counter, guaranteeing identical episode sequences across runs with the same configuration.

All experimental artefacts were systematically archived:

- **Model checkpoints:** both the best-performing model (`best.pt`) and the final training checkpoint (`final_checkpoint.pt`) were saved for every run.
- **Training metrics:** logged every epoch and stored as NumPy archives (`.npz`). These files were used to generate all figures presented in Chapter 9.
- **Plots:** all training and evaluation curves were exported as both PNG and PDF files are stored alongside the metrics.
- **Run configuration:** the complete set of hyperparameters and environment settings was written to a structured text file within the experiment directory, providing a full record of each run.

Chapter 9

Results

The experimental protocol and evaluation metrics are described in Chapter 8. All figures show the median test reward computed over a range of complexities surrounding the training difficulty. The shaded bands span the inter-quartile and full range across the tested complexities, and the deterministic and stochastic test lines refer to point evaluations at the exact training complexity.

9.1 Algorithm Comparison: REINFORCE vs. PPO

PPO reached higher median test rewards than REINFORCE (Figure 9.1) and was selected for all later experiments.

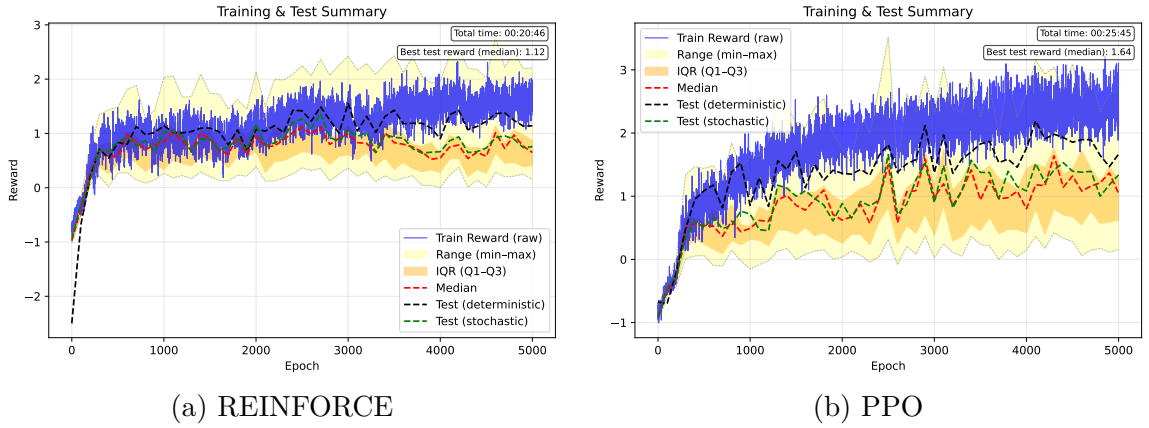
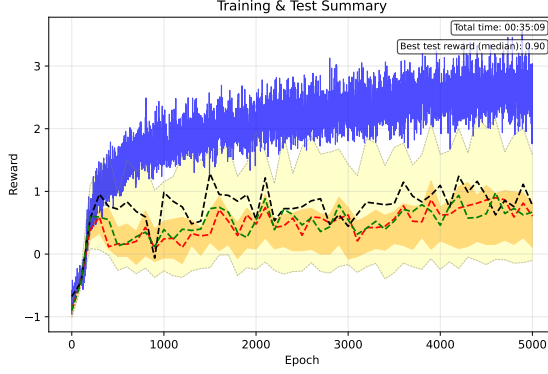


Figure 9.1: REINFORCE vs PPO on the basic task (training complexity 0.5).

In all subsequent reward figures, the legend is identical to that of Figure 9.1 and is therefore omitted.

9.2 PPO Hyperparameter Tuning

The sweeps described in Section 8.1 yielded the final configuration: 1 intra-epoch, mini-batch size 64, parallel batch size 64, Adam optimiser, and learning rate 6×10^{-4} . Only the most informative comparisons are shown below.

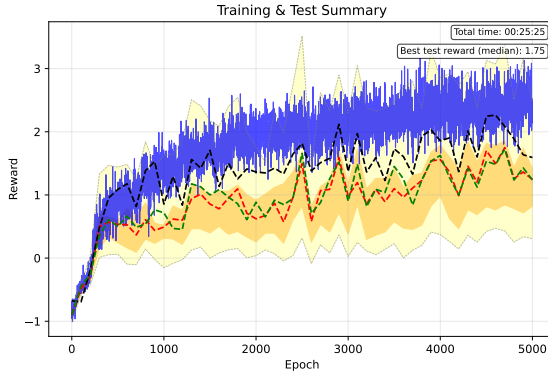


(a) Intra-epochs = 2

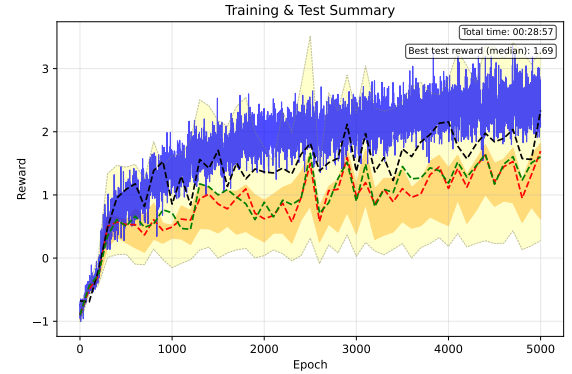


(b) Intra-epochs = 4

Figure 9.2: Intra-epochs = 1 (Figure 9.1 (b)) had the highest median and was retained.

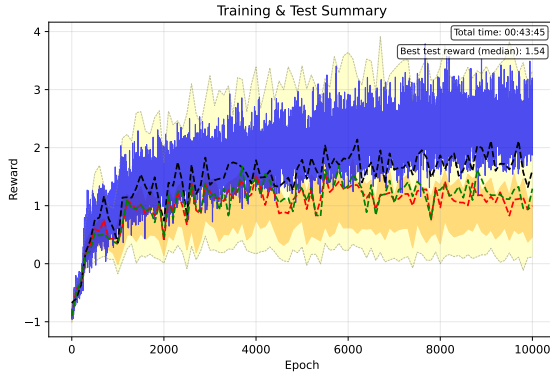


(a) Mini-batch size 64

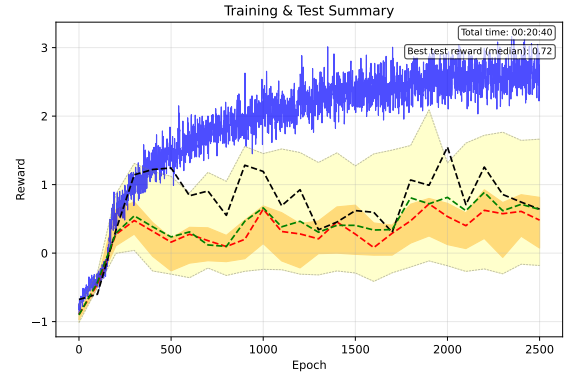


(b) Mini-batch size 1600

Figure 9.3: Mini-batch size 64 (a) gave the highest median and was retained. Size 256 (not shown) had a median test value of 1.67.

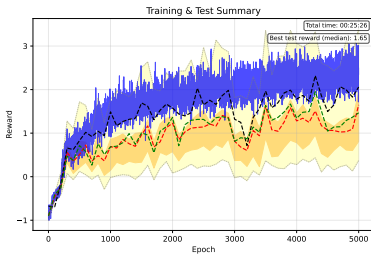


(a) Batch size 32 (10 000 epochs)

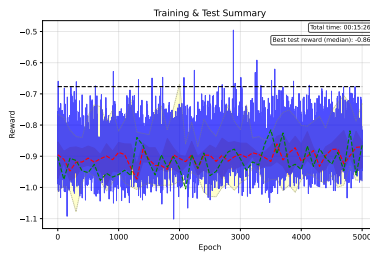


(b) Batch size 128 (2 500 epochs)

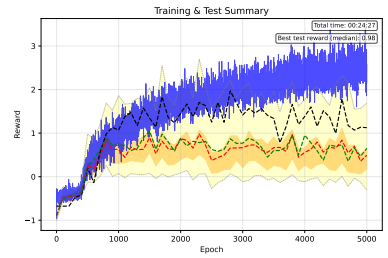
Figure 9.4: Batch size 64 (Figure 9.3 (a)) had the highest median and was retained.



(a) AdamW



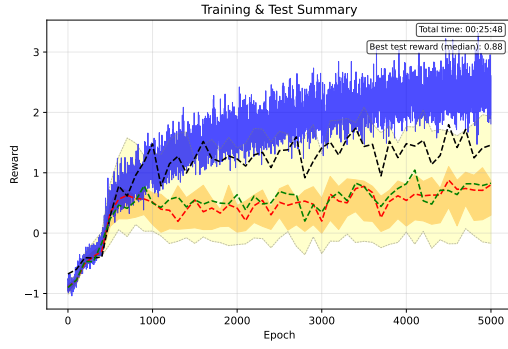
(b) SGD



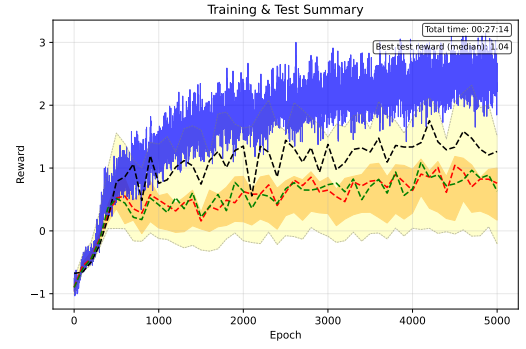
(c) RMSprop

Figure 9.5: Adam (Figure 9.3 (a)) had the highest median and was retained. AdamW performed similarly, while SGD and RMSprop were inferior.

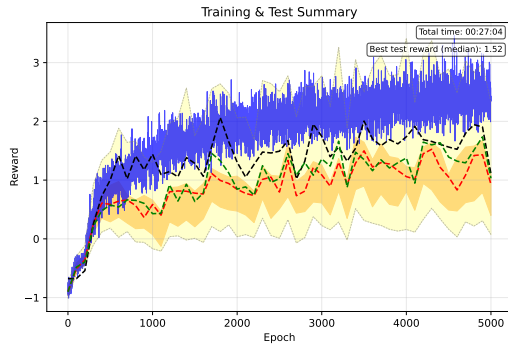
Both the batch size experiment and the optimiser experiment retained default values. The last experiment is the learning rate experiment.



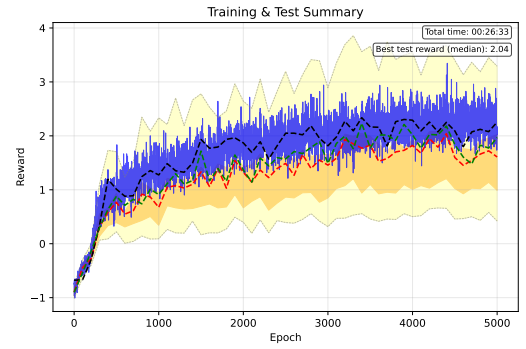
(a) $\text{LR} = 10^{-4}$



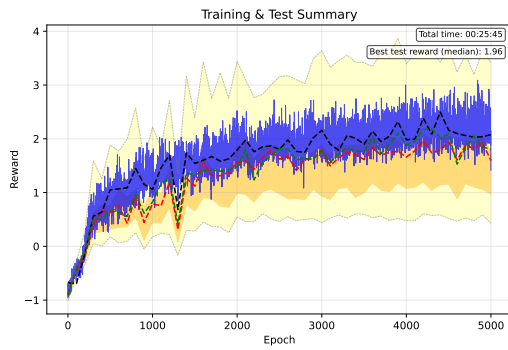
(b) $\text{LR} = 2 \times 10^{-4}$



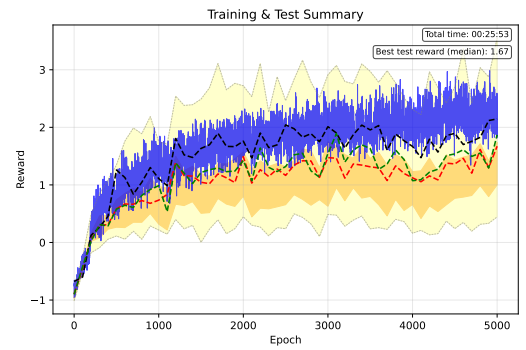
(c) $\text{LR} = 4 \times 10^{-4}$



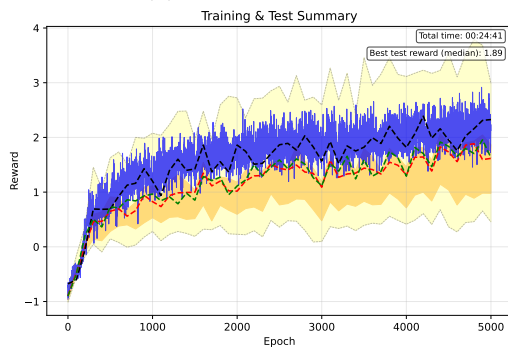
(d) $\text{LR} = 6 \times 10^{-4}$



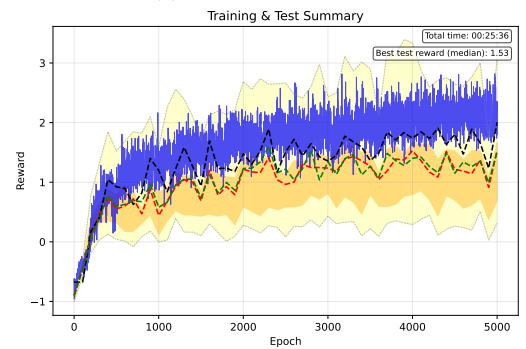
(e) $\text{LR} = 7 \times 10^{-4}$



(f) $\text{LR} = 8 \times 10^{-4}$



(g) $\text{LR} = 9 \times 10^{-4}$



(h) $\text{LR} = 10^{-3}$

Figure 9.6: Learning rate experiment. 6×10^{-4} (d) achieved the highest median and was retained. The baseline 5×10^{-4} is shown in Figure 9.3 (a); 3×10^{-4} (not shown) had median 1.35.

9.3 Scaling to Larger Grids

Moving to a 19×19 grid with dynamic complexity on the basic task, auxiliary prediction heads enabled the agent to sustain higher complexity levels (Figure 9.7 (d)). In dynamic curricula, the reward signal is unreliable; the highest sustained complexity without regressing was therefore used to compare configurations in all subsequent experiments.

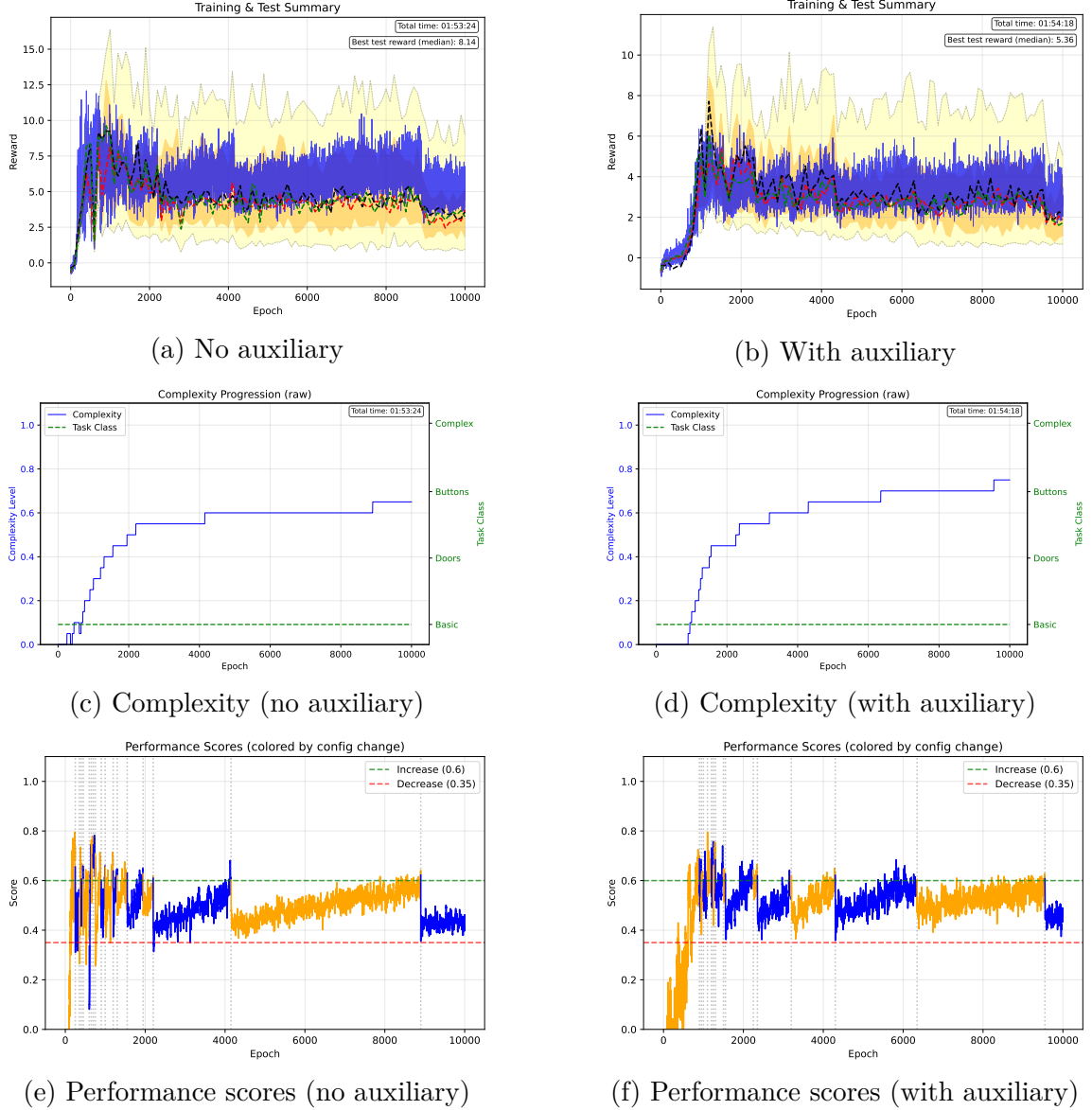
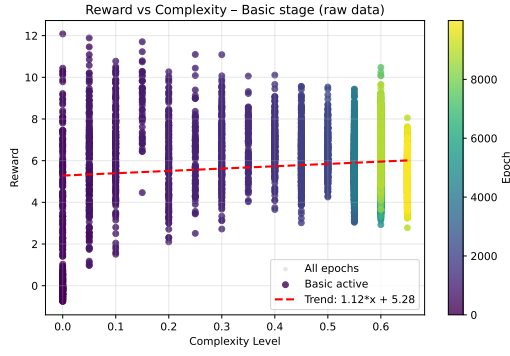
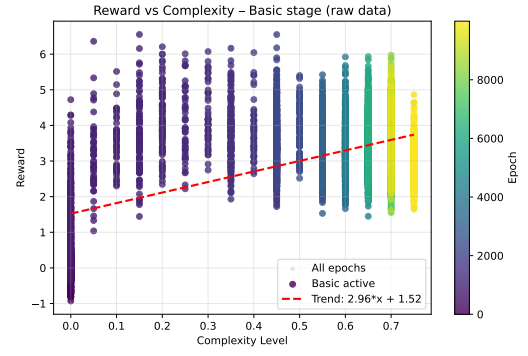


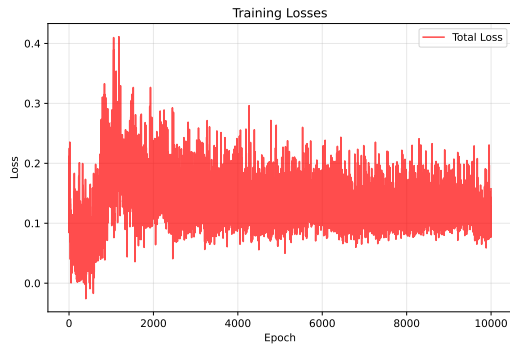
Figure 9.7: Training curves for the basic task on a 19×19 grid without and with auxiliary tasks.



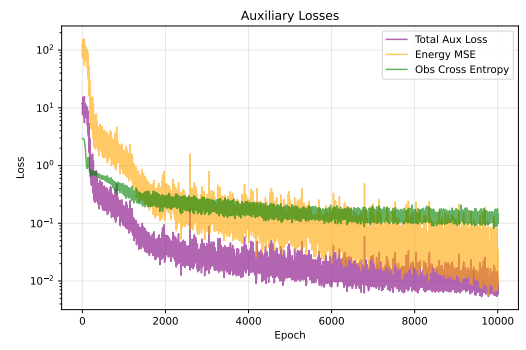
(a) No auxiliary



(b) With auxiliary



(c) Policy loss (auxiliary run)



(d) Auxiliary losses (auxiliary run)

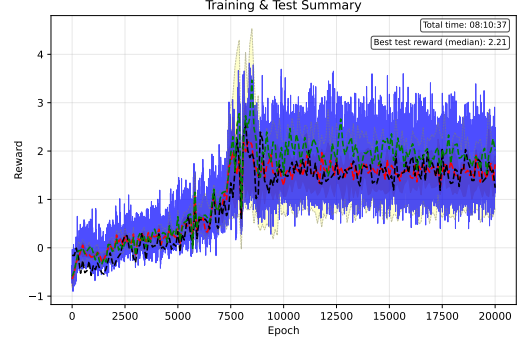
Figure 9.8: Reward versus complexity during the basic stage (top) and loss curves for the auxiliary configuration (bottom).

9.4 Network Architecture Comparison

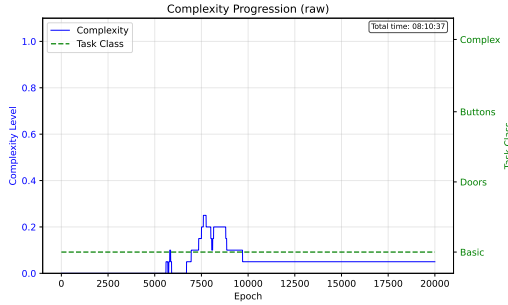
Three architectures were evaluated on the 19×19 basic dynamic curriculum with auxiliary tasks: an LSTM with hidden size 768 (learning rate 6×10^{-4}), a Transformer with the same learning rate, and a Transformer with a reduced learning rate of 1×10^{-4} (as the higher learning rate did not converge). Only the LSTM successfully sustained high complexity levels (Figure 9.10 (b)); the Transformer configurations plateaued early and could not match the LSTM’s final performance. All subsequent experiments, therefore, used the LSTM.



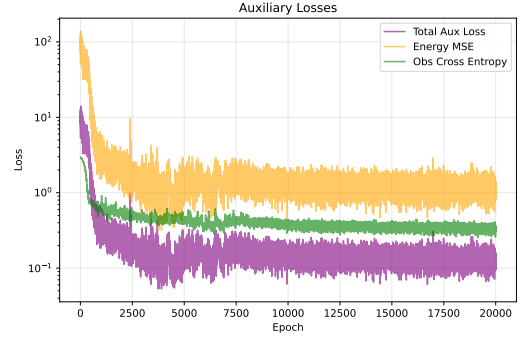
(a) Transformer, $LR = 6 \times 10^{-4}$



(b) Transformer, $LR = 1 \times 10^{-4}$



(c) Complexity ($LR = 1 \times 10^{-4}$)

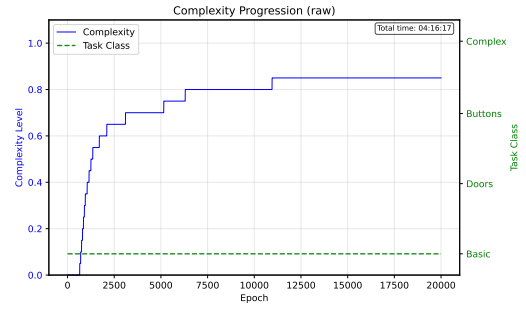


(d) Auxiliary losses ($LR = 1 \times 10^{-4}$)

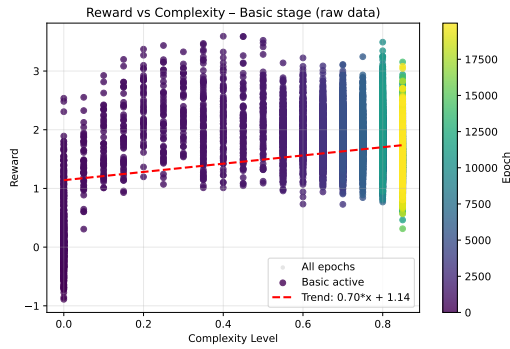
Figure 9.9: Transformer runs. Both learning rates failed to reach high complexity.



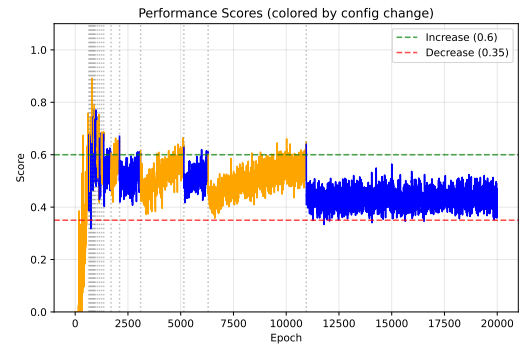
(a) Rewards



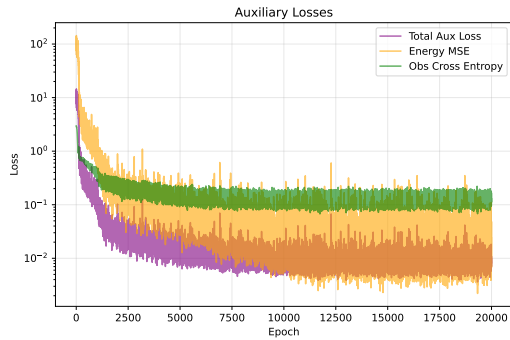
(b) Complexity progression



(c) Reward vs complexity



(d) Performance scores



(e) Auxiliary losses



(f) Policy loss

Figure 9.10: LSTM (hidden size 768). The agent's achieved complexity level increased while auxiliary and policy learning remained stable.

9.5 Task Curriculum Comparison

The full curriculum (basic → doors → buttons → complex) showed that the agent could not master the hardest mixed door-button environment. The doors-only and buttons-only runs are shown in Figures 9.11 and 9.12; the full curriculum appears in Figure 9.13.

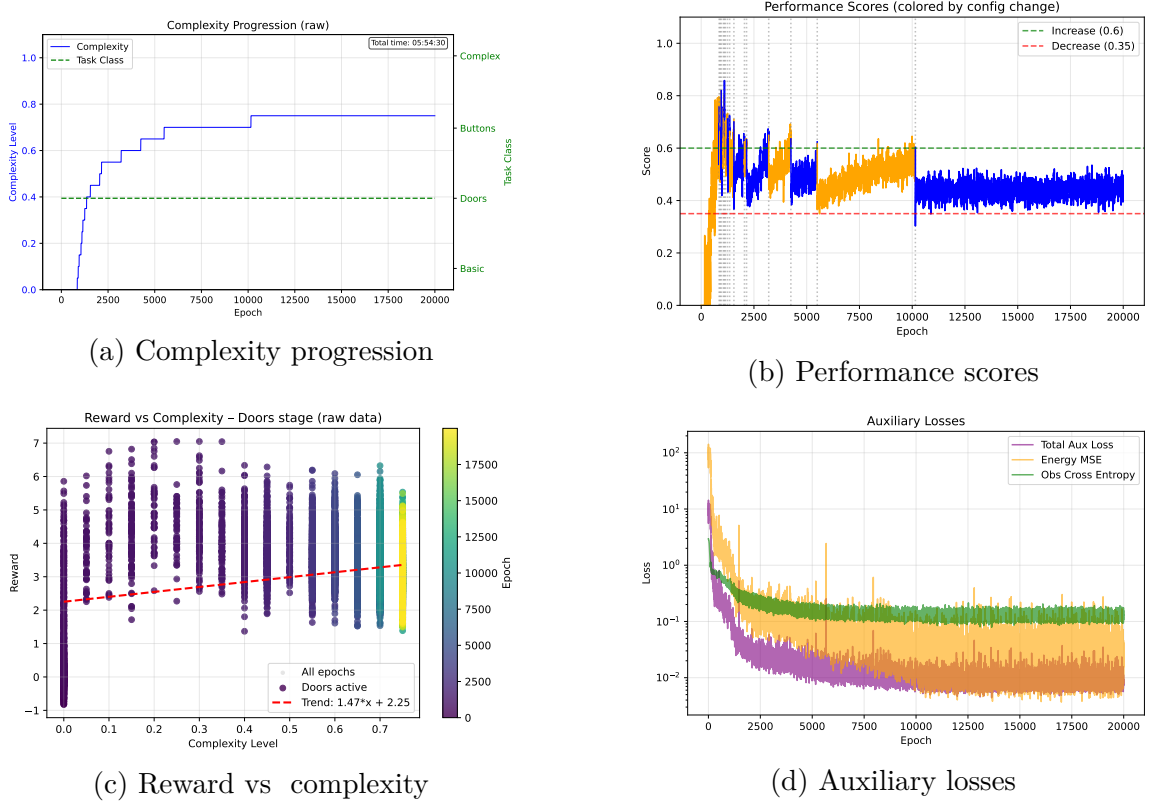
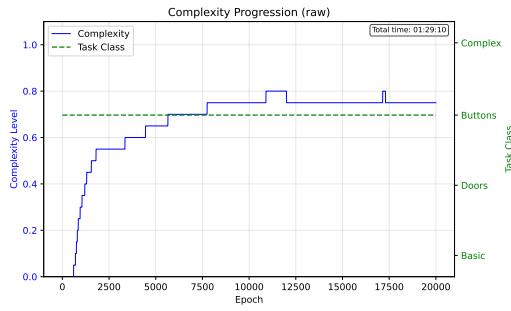
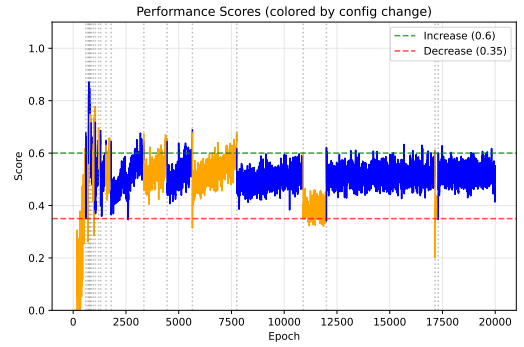


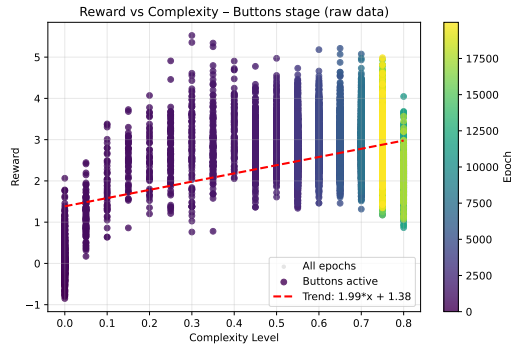
Figure 9.11: Doors-only curriculum. The agent successfully learned the doors task, though its performance did not reach the same level of proficiency as on the basic task.



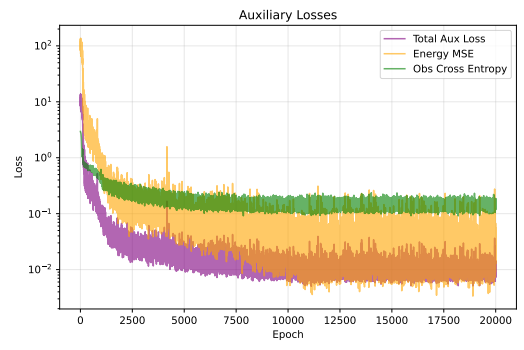
(a) Complexity progression



(b) Performance scores

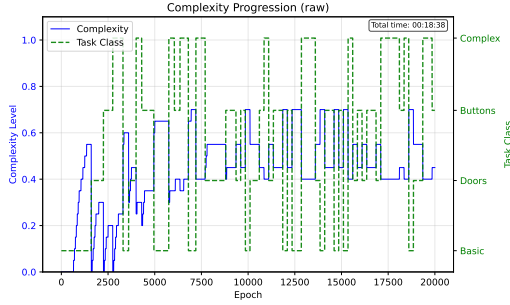


(c) Reward vs complexity

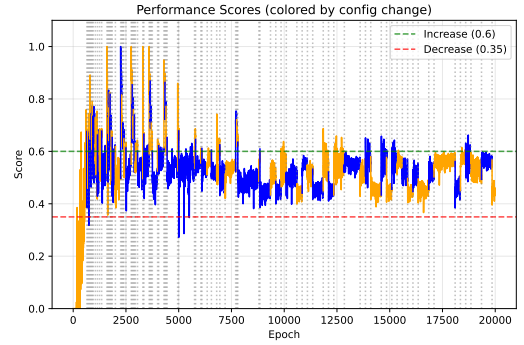


(d) Auxiliary losses

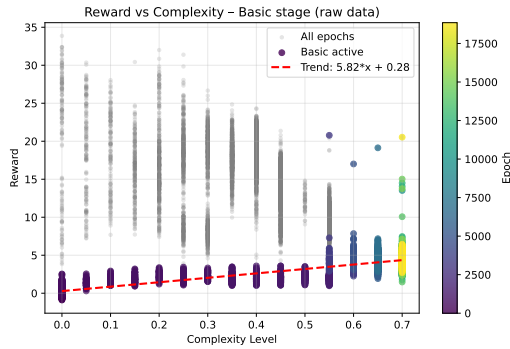
Figure 9.12: Buttons-only curriculum. The agent plateaued, similarly to the doors experiment.



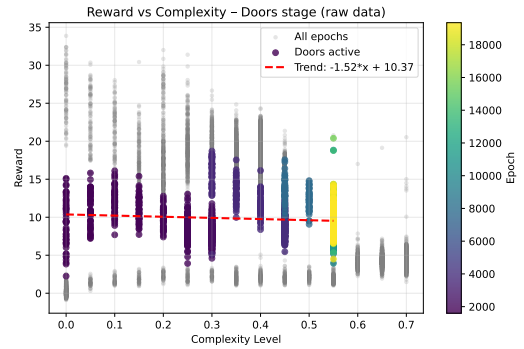
(a) Complexity progression



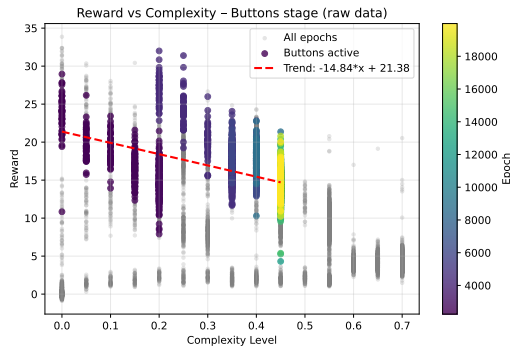
(b) Performance scores



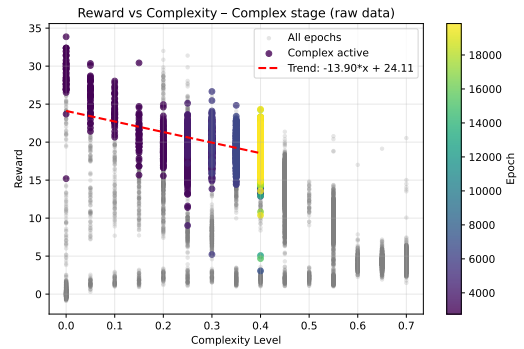
(c) Reward vs complexity (basic stage)



(d) Reward vs complexity (doors stage)



(e) Reward vs complexity (buttons stage)



(f) Reward vs complexity (complex stage)

Figure 9.13: Full curriculum. The agent could not master any stage. Only the basic stage achieved solid results.

Chapter 10

Future Work and Extensions

The present work establishes a controlled, procedurally generated, partially observable environment, a dynamic curriculum manager, and baseline LSTM and Transformer agents that together form a robust framework for memory research in reinforcement learning. Because the entire codebase—from the C++ environment core to the Python training loop—was designed with extensibility in mind, many directions for deepening the investigation and broadening the applicability are immediately accessible. This chapter describes concrete extensions that build directly on the existing implementation, tools, and experimental protocols.

10.1 Environment Enhancements

Multi-Agent and Cooperative Extensions

The `GridMazeWorld` class is stateless with respect to the agent count; it could be cloned internally to support two or more agents sharing the same grid. Cooperative memory tasks—such as one agent holding a button while the other passes a door, or jointly mapping a maze—would test inter-agent communication and shared memory. The vectorised environment wrapper already handles multiple independent environments; the same wrapper can be adapted to step multiple agents in a single environment instance, returning a list of per-agent observations and rewards.

Dynamic Obstacles and Adversarial Entities

Introducing moving obstacles (e.g., guards following predefined patrol routes or performing random walks) would force the agent to update its spatial memory continuously. The dynamics can be added to the `step()` function by updating a set of dynamic obstacle positions after the agent acts. Because the grid is small and the step function is already optimised, this would incur only a marginal overhead while adding a rich temporal prediction challenge that the current auxiliary tasks (next-energy and next-observation prediction) only partially address.

More Complex Button–Door Logic

The current button–door coupling is simple: a door may require one button on each side, and a button breaks probabilistically. The `Button` and `door` classes can be extended to support logical formulas over multiple buttons, e.g.:

- **AND** gates: all assigned buttons must be pressed (possibly within a time window).
- **OR** gates: any assigned button opens the door.
- **Toggle** doors: each press flips the door state, requiring the agent to remember the current state from past interaction.
- **Timed failure**: a button may become temporarily disabled after use, adding a short-term planning horizon.

These logical extensions directly probe relational memory and the agent’s ability to track discrete, non-local dependencies.

Richer Observation Spaces

The current 3×3 token observation is intentionally minimal. The Python wrapper already calls a `render_obs()` method; extending it to return an egocentric 5×5 symbolic image, a small RGB rendering, or even an audio cue (e.g., a sound when a distant door opens) would enable studying multimodal memory integration. Language-grounded instructions, similar to BabyAI, could be added by associating tasks with a small synthetic grammar and providing the instruction as an additional token sequence at the start of an episode.

Memory-Probe Episodes

Inspired by Psychlab and diagnostic benchmarks, one could implement ‘probe’ episodes: after an exploration phase, the agent is asked a question about the environment (e.g., “Which door did the button that was in the upper-right room belong to?”) and receives a reward based on the answer. This would provide a direct measure of the agent’s internal representations’ fidelity without relying solely on rewards from navigation and food collection. The environment already maintains ground-truth metadata (door open/closed, button working/broken, food regrowth timers) that could be used to generate such probes.

Teleportation

A simple but cognitively demanding extension is to randomly teleport the agent to a different location on the grid. The environment already has a `soft_reset()` function that could be simplified to reset only the agent’s position instead of reinitialising the whole grid. Teleportation forces the agent to re-orient itself purely from recent observations and memory, challenging its ability to maintain a coherent spatial map despite sudden viewpoint shifts. This directly tests the core memory hypothesis of the thesis and can be combined with the dynamic complexity manager to increase the teleportation frequency or distance as performance improves gradually.

10.2 Architectures and State Representation Learning

Structured Spatial Memories

Because the task demands building an internal map, dedicated spatial memory architectures (e.g., Neural Map) can be realised by allocating a 2D grid that is written to using the agent’s (latent or estimated) pose and read via differentiable attention. The auxiliary observation-prediction head already encourages spatial awareness; an explicit map layer with a reconstruction loss for visited cells could accelerate the formation of allocentric spatial knowledge.

Hybrid Recurrence–Transformer Models

The current Transformer agent uses a fixed set of learnable memory tokens that persist across forward calls. Preliminary experiments showed under-attention to recent observations. Future work can refine this by:

- Increasing the number of memory tokens and adding a learned gating mechanism that decides what to write and forget.
- Combining a lightweight recurrent encoder (e.g., a small LSTM) that compresses the observation stream into a compact state, which is then used as additional prefix tokens in the Transformer.
- Comparing with modern state-space models (e.g., Mamba [20]) that promise long-range memory at lower cost.

A systematic comparison under identical curriculum and seed conditions would reveal which hybrid best balances temporal credit assignment and computational budget.

Improved Auxiliary Prediction Tasks

The auxiliary losses (next-energy and next-observation prediction) proved beneficial in scaling experiments. These can be extended:

- **Contrastive objectives:** encourage the latent state to be invariant to irrelevant details (e.g., exact position of a depleted food source) while retaining information about doors and buttons.
- **Door-state prediction:** add a head that predicts whether each door will be open or closed k steps into the future. The environment can provide this ground truth without additional simulation cost.
- **Button-working prediction:** a binary classification head predicting whether a recently pressed button is still functional, directly incentivising the agent to track button breakage events.

All these targets are cheap to compute and can be appended to the existing auxiliary loss vector.

Controlled Forgetting and Capacity Limits

One can systematically limit the LSTM hidden size or the number of Transformer memory tokens to study the effect of capacity constraints. A forgetting auxiliary loss—minimising the mutual information between the hidden state and outdated observations—can be implemented by training a discriminator to predict the age of stored information. This may reveal whether agents develop selective forgetting strategies and whether they can re-explore areas efficiently after a long delay.

10.3 Training Algorithms and Curriculum Design

Refined Performance Metrics for the Dynamic Curriculum

The current curriculum manager (`dynamic_complexity.py`) adjusts complexity based on average reward relative to a stage-specific best. This can be augmented with:

- **Exploration coverage:** the fraction of reachable cells visited at least once.
- **Door-opening success rate:** the proportion of door encounters that the agent successfully resolves.

A multi-criteria decision rule would prevent the curriculum from stalling when the agent discovers a locally optimal policy (e.g., staying near a food cluster) that yields high reward but ignores challenging areas.

Meta-Learning the Curriculum

Instead of a fixed progression of task classes, a meta-controller could learn to select the next complexity or task class to maximise long-term progress. The dynamic complexity manager already has access to the agent’s recent performance; one could train a small neural network (the “teacher”) whose action space is the discrete set of complexity adjustments, using the agent’s final return as reward. This would automate the discovery of optimal training schedules.

World Models and Latent Planning

Model-based agents (e.g., DreamerV2 [22]) learn a latent dynamics model and plan in latent space. Because GridMazeWorld runs extremely fast, a world-model agent could be trained entirely within the C++ loop with minimal overhead. This would enable a direct comparison between model-based and model-free memory use under identical observability and reward structures, a study currently impractical in high-fidelity 3D simulators.

10.4 Evaluation, Diagnostics, and Reproducibility

Memory Probing and Representation Analysis

The trained agents’ hidden states can be systematically probed post-hoc. For example, linear decoders can be trained to predict the agent’s global (x,y) position, the open/closed status of each door, and which buttons have been pressed, using only the LSTM hidden state or the Transformer’s memory tokens as input. This would reveal whether the agent has formed an allocentric map or merely reacts to local cues.

Failure Mode Classification

The environment can automatically categorise episodes ending in zero reward. The environment already returns per-episode metadata in the info dictionary: cumulative food collected, number of door openings, number of button presses, energy at death, etc. Aggregating these statistics across thousands of episodes would quantify whether failures stem from energy exhaustion, being trapped behind a closed door, or failing to press the correct button—giving direct insight into memory failures.

Generalisation to Unseen Maze Generators

The experimental protocol can be extended with a hold-out set of maze generators that differ in obstacle density, door–button graph topology, or grid size. Zero-shot performance on these unseen configurations directly tests whether the agent has learned general relational reasoning or overfitted to the training distribution. The current training script already accepts a test environment configuration; adding a separate configurable test suite is straightforward.

Interactive Visualisation Dashboard

A real-time web-based dashboard (using Dash or Streamlit) could display the agent’s current observation, its hidden state projected to 2D via PCA, the Transformer’s attention weights, predicted next observations, and the dynamic curriculum state. The existing Python environment wrapper renders the ground-truth grid; overlaying the agent’s internal beliefs (e.g., decoded map) would greatly accelerate debugging and qualitative understanding.

Containerised Benchmarks and Leaderboards

To ensure exact reproducibility, the entire pipeline can be packaged in a Docker image with pinned Python dependencies and a compiled C++ environment. A public leaderboard, similar to POPGym, could be maintained where researchers submit their agent’s performance on standardised GridMazeWorld tasks (e.g., `complex`, grid size 19, fixed seed set).

10.5 Real-World Transfer

A physical robot (e.g., a TurtleBot with a downward-facing camera) could navigate a real-world maze marked with fiducial markers representing doors and buttons. The agent’s policy, trained exclusively in GridMazeWorld, could be transferred by providing an embedding of camera observations that maps detected markers to the discrete token representation. This would test the robustness of learned memory strategies to sensor noise and continuous dynamics, and the lightweight simulation makes rapid policy iteration possible before expensive real-world tests.

Chapter 11

Conclusion

This thesis set out to advance the understanding of memory mechanisms in reinforcement learning under partial observability. The principal contributions are:

1. **GridMazeWorld: a new memory-focused benchmark.** I designed and implemented a procedurally generated, partially observable 2D grid world that integrates spatial navigation, relational dependencies (button-door logic with non-local effects), temporal dynamics (periodic doors, food regrowth), and resource management (energy budget) into a single environment. The C++ implementation achieves hundreds of thousands of steps per second on a single CPU core, making it suitable for large-scale deep RL experiments.
2. **Principled generation algorithms.** The environment uses articulation-point analysis to guarantee map connectivity while placing obstacles, doors, and buttons in computationally efficient, deterministic passes. This yields a wide variety of topologies without sacrificing consistency or speed.
3. **Dynamic curriculum manager.** A self-adapting curriculum was developed that adjusts the environment’s complexity level and task class online based on the agent’s performance. This manager avoids manual scheduling and was shown to be essential for scaling to larger mazes and mixed task classes.
4. **Baseline agent architectures and auxiliary tasks.** I implemented LSTM and Transformer agents with configurable auxiliary self-supervised prediction heads (next-energy, next-observation). The entire training pipeline supports both REINFORCE and PPO with vectorised data collection.
5. **Systematic empirical study.** Through carefully controlled hyperparameter sweeps and ablation studies, I quantified the impact of network architecture, curriculum design, and auxiliary supervision on memory-demanding tasks.

11.1 Key Findings

- **Dynamic curriculum is indispensable for hard problems.** Training directly on high-complexity or mixed-task configurations resulted in slow or stalled learning. A curriculum that first taught survival and then other challenges allowed the agent to achieve better results on more complex tasks.
- **Auxiliary prediction losses improve scaling.** Adding simple next-energy and next-observation regression heads boosted test performance, especially when the reward signal was sparse. This confirms that shaping the latent representation with environment-dynamics objectives is beneficial.
- **LSTM remains a strong baseline, but Transformer performance depends on context handling.** Under comparable conditions, the LSTM agent outperformed the Transformer agent. Transformer probably did not fully exploit the most recent observation tokens, indicating that careful design of memory tokens and positional encoding is required for competitive long-context memory.
- **GridMazeWorld fills a gap between simple grid-worlds and high-fidelity 3D simulators.** Most existing memory benchmarks either isolate a single memory modality (T-maze, MemoryGym) or are computationally expensive (Obstacle Tower). GridMazeWorld simultaneously demands spatial, sequential, relational, and temporal memory while remaining fast enough for hundreds of millions of interactions. This makes it an effective proxy for evaluating network architectures before committing to expensive 3D environments.

11.2 Final Remarks

The message of this work is that memory in reinforcement learning is not a monolithic capability but a composite of multiple, interdependent demands. A well-designed benchmark must reflect this by integrating spatial, relational, temporal, and resource constraints into a single, partially observable setting. GridMazeWorld was built precisely to fill that role, replacing primitive isolated memory tests with a rich, computationally lightweight testbed that can quickly reveal the strengths and weaknesses of a memory architecture.

By making the environment highly configurable and pairing it with an automatic curriculum, the framework enables a rapid iterate-and-validate cycle: promising network designs can be cheaply prototyped and tested in GridMazeWorld, and only the most successful and large ones need to be scaled up to 3D simulators. This workflow, combined with the detailed diagnostics and extensibility described in the previous chapter, positions GridMazeWorld as a lasting contribution to the study of memory in reinforcement learning.

The source code, trained models, and full experiment logs are publicly available at <https://github.com/Samuel-2000/Masters-Thesis-REINFORCED-LEARNING-OF-AGENTS-WITH-MEMORY>, providing a reproducible foundation for future research.

Bibliography

- [1] *Causal attention mask* https://llm.modular.com/step_03.html.
- [2] *LSTM cell* https://d2l.ai/chapter_recurrent-modern/lstm.html.
- [3] *Positional encoding in a Transformer*
<https://www.geeksforgeeks.org/nlp/positional-encoding-in-transformers/>.
- [4] *RNN unrolled*
<https://necst.it/wp-content/uploads/2018/05/RNN-unrolled.png>.
- [5] *T-maze* <https://www.youtube.com/watch?v=cL7H5Sxw1KM>.
- [6] AGARWAL, P.; RAHMAN, A. A.; ST CHARLES, P.-L.; PRINCE, S. J. D. and KAHOU, S. E. *Transformers in Reinforcement Learning: A Survey*. 2023. Available at: <https://arxiv.org/abs/2307.05979>.
- [7] ARDHIAN, A. and FANANY, M. I. Learning Explicit and Implicit Knowledge with Differentiable Neural Computer. In:. October 2017.
- [8] BEATTIE, C.; LEIBO, J. Z.; TEPLYASHIN, D.; WARD, T.; WAINWRIGHT, M. et al. *DeepMind Lab*. 2016. Available at: <https://arxiv.org/abs/1612.03801>.
- [9] BLUMENTHAL, O. and SHANI, G. Domain independent heuristics for online stochastic contingent planning. *Annals of Mathematics and Artificial Intelligence*, july 2024, vol. 93, p. 519–540.
- [10] CHEN, L.; LU, K.; RAJESWARAN, A.; LEE, K.; GROVER, A. et al. *Decision Transformer: Reinforcement Learning via Sequence Modeling*. 2021. Available at: <https://arxiv.org/abs/2106.01345>.
- [11] CHEREPANOV, E.; KACHAEV, N.; KOVALEV, A. K. and PANOV, A. I. *Memory, Benchmark & Robots: A Benchmark for Solving Complex Tasks with Reinforcement Learning*. 2026. Available at: <https://arxiv.org/abs/2502.10550>.
- [12] CHEVALIER BOISVERT, M.; BAHDANAU, D.; LAHLOU, S.; WILLEMS, L.; SAHARIA, C. et al. *BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning*. 2019. Available at: <https://arxiv.org/abs/1810.08272>.
- [13] CHEVALIER BOISVERT, M.; DAI, B.; TOWERS, M.; LAZCANO, R. de; WILLEMS, L. et al. *Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks*. 2023. Available at: <https://arxiv.org/abs/2306.13831>.

- [14] CHO, K.; MERRIENBOER, B. van; GULCEHRE, C.; BAHDANAU, D.; BOUGARES, F. et al. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. 2014. Available at: <https://arxiv.org/abs/1406.1078>.
- [15] COBBE, K.; HESSE, C.; HILTON, J. and SCHULMAN, J. *Leveraging Procedural Generation to Benchmark Reinforcement Learning*. 2020. Available at: <https://arxiv.org/abs/1912.01588>.
- [16] ECHCHAHED, A. and CASTRO, P. S. *A Survey of State Representation Learning for Deep Reinforcement Learning*. 2025. Available at: <https://arxiv.org/abs/2506.17518>.
- [17] ESPEHOLT, L.; SOYER, H.; MUNOS, R.; SIMONYAN, K.; MNIH, V. et al. *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*. 2018. Available at: <https://arxiv.org/abs/1802.01561>.
- [18] GRAVES, A.; WAYNE, G. and DANIHELKA, I. *Neural Turing Machines*. 2014. Available at: <https://arxiv.org/abs/1410.5401>.
- [19] GRAVES, A.; WAYNE, G.; REYNOLDS, M.; HARLEY, T.; DANIHELKA, I. et al. Hybrid Computing Using a Neural Network with Dynamic External Memory. *Nature*, 2016, vol. 538, no. 7626, p. 471–476. Available at: <https://pubmed.ncbi.nlm.nih.gov/27732574/>.
- [20] GU, A. and DAO, T. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. Available at: <https://arxiv.org/abs/2312.00752>.
- [21] HAFNER, D.; LILLICRAP, T.; FISCHER, I.; VILLEGAS, R.; HA, D. et al. *Learning Latent Dynamics for Planning from Pixels*. 2019. Available at: <https://arxiv.org/abs/1811.04551>.
- [22] HAFNER, D.; LILLICRAP, T.; NOROUZI, M. and BA, J. *Mastering Atari with Discrete World Models*. 2022. Available at: <https://arxiv.org/abs/2010.02193>.
- [23] HANSEN, E. A. Finite-Memory Control of Partially Observable Systems. *Machine Learning*, 1999, vol. 35, no. 1, p. 1–39. Available at: <https://www.proquest.com/docview/304430723?pq-origsite=gscholar&fromopenview=true&sourcetype=Dissertations%20&%20Theses>.
- [24] HOCHREITER, S. and SCHMIDHUBER, J. Long Short-Term Memory. *Neural Computation*, november 1997, vol. 9, p. 1735–1780. Available at: https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory.
- [25] HU, H.; YE, J.; ZHU, G.; REN, Z. and ZHANG, C. Generalizable Episodic Memory for Deep Reinforcement Learning. In: *Proceedings of Machine Learning Research (PMLR)*. 2021. Available at: <https://proceedings.mlr.press/v139/hu21d/hu21d.pdf>.
- [26] HUANG, N.-C.; HSIEH, P.-C.; HO, K.-H. and WU, I.-C. *PPO-Clip Attains Global Optimality: Towards Deeper Understandings of Clipping*. 2024. Available at: <https://arxiv.org/abs/2312.12065>.
- [27] JULIANI, A.; KHALIFA, A.; BERGES, V.-P.; HARPER, J.; TENG, E. et al. *Obstacle Tower: A Generalization Challenge in Vision, Control, and Planning*. 2019. Available at: <https://arxiv.org/abs/1902.01378>.

- [28] Kaelbling, L. P.; Littman, M. L. and Cassandra, A. R. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 1998, vol. 101. Available at: <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf>.
- [29] Kaelbling, L. P.; Littman, M. L. and Moore, A. W. Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 1996, vol. 4, p. 237–285. Available at: <https://www.jair.org/index.php/jair/article/view/10166>.
- [30] Kapturowski, S.; Ostrovski, G.; Quan, J.; Munos, R. and Dabney, W. Recurrent Experience Replay in Distributed Reinforcement Learning. In: *International Conference on Learning Representations (ICLR)*. 2019. Available at: <https://openreview.net/pdf?id=rilyTjAqYX>.
- [31] Kempka, M.; Wydmuch, M.; Runc, G.; Toczek, J. and Jaśkowski, W. *ViZDoom: A Doom-based AI Research Platform for Visual Reinforcement Learning*. 2016. Available at: <https://arxiv.org/abs/1605.02097>.
- [32] Küttler, H.; Nardelli, N.; Miller, A. H.; Raileanu, R.; Selvatichi, M. et al. *The NetHack Learning Environment*. 2020. Available at: <https://arxiv.org/abs/2006.13760>.
- [33] Laskin, M.; Lee, K.; Stooke, A.; Pinto, L.; Abbeel, P. et al. *Reinforcement Learning with Augmented Data*. 2020. Available at: <https://arxiv.org/abs/2004.14990>.
- [34] Leibo, J. Z.; Masson d’Autume, C. de; Zoran, D.; Amos, D.; Beattie, C. et al. *Psychlab: A Psychology Laboratory for Deep Reinforcement Learning Agents*. 2018. Available at: <https://arxiv.org/abs/1801.08116>.
- [35] Mao, H.; Zhao, R.; Chen, H.; Hao, J.; Chen, Y. et al. *Transformer in Transformer as Backbone for Deep Reinforcement Learning*. 2023. Available at: <https://arxiv.org/abs/2212.14538>.
- [36] Morad, S.; Kortvelesy, R.; Liwicki, S. and Prorok, A. *Reinforcement Learning with Fast and Forgetful Memory*. 2023. Available at: <https://arxiv.org/abs/2310.04128>.
- [37] Ni, T.; Eysenbach, B. and Salakhutdinov, R. *Recurrent Model-Free RL Can Be a Strong Baseline for Many POMDPs*. 2022. Available at: <https://arxiv.org/abs/2110.05038>.
- [38] Parisotto, E. and Salakhutdinov, R. *Neural Map: Structured Memory for Deep Reinforcement Learning*. 2017. Available at: <https://arxiv.org/abs/1702.08360>.
- [39] Parisotto, E.; Song, H. F.; Rae, J. W.; Pascanu, R.; Gulcehre, C. et al. *Stabilizing Transformers for Reinforcement Learning*. 2019. Available at: <https://arxiv.org/abs/1910.06764>.
- [40] Pineau, J.; Gordon, G. and Thrun, S. Point-based Value Iteration: An Anytime Algorithm for POMDPs. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2003, p. 1025–1032. Available at: <https://www.ijcai.org/Proceedings/03/Papers/147.pdf>.

- [41] PLEINES, M.; PALLASCH, M.; ZIMMER, F. and PREUSS, M. *Memory Gym: Towards Endless Tasks to Benchmark Memory Capabilities of Agents*. 2025. Available at: <https://arxiv.org/abs/2309.17207>.
- [42] RAE, J. W.; POTAPENKO, A.; JAYAKUMAR, S. M. and LILLICRAP, T. P. *Compressive Transformers for Long-Range Sequence Modelling*. 2019. Available at: <https://arxiv.org/abs/1911.05507>.
- [43] SCHULMAN, J.; MORITZ, P.; LEVINE, S.; JORDAN, M. and ABBEEL, P. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2018. Available at: <https://arxiv.org/abs/1506.02438>.
- [44] SCHULMAN, J.; WOLSKI, F.; DHARIWAL, P.; RADFORD, A. and KLIMOV, O. *Proximal Policy Optimization Algorithms*. 2017. Available at: <https://arxiv.org/abs/1707.06347>.
- [45] SHAO, Z.; WANG, P.; ZHU, Q.; XU, R.; SONG, J. et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. Available at: <https://arxiv.org/abs/2402.03300>.
- [46] SHRIDHAR, M.; THOMASON, J.; GORDON, D.; BISK, Y.; HAN, W. et al. *ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks*. 2020. Available at: <https://arxiv.org/abs/1912.01734>.
- [47] STANIĆ, A.; TANG, Y.; HA, D. and SCHMIDHUBER, J. *Learning to Generalize with Object-centric Agents in the Open World Survival Game Crafter*. 2022. Available at: <https://arxiv.org/abs/2208.03374>.
- [48] SUKHBAATAR, S.; SZLAM, A.; SYNNAEVE, G.; CHINTALA, S. and FERGUS, R. *MazeBase: A Sandbox for Learning from Games*. 2016. Available at: <https://arxiv.org/abs/1511.07401>.
- [49] SUTTON, R. S. and BARTO, A. G. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. Available at: <http://incompleteideas.net/book/the-book.html>.
- [50] TAO, R. Y.; GUO, K.; ALLEN, C. and KONIDARIS, G. *Benchmarking Partial Observability in Reinforcement Learning with a Suite of Memory-Improvable Domains*. 2025. Available at: <https://arxiv.org/abs/2508.00046>.
- [51] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L. et al. *Attention Is All You Need*. 2023. Available at: <https://arxiv.org/abs/1706.03762>.
- [52] ZHU*, G.; LIN*, Z.; YANG, G. and ZHANG, C. Episodic Reinforcement Learning with Associative Memory. In: *International Conference on Learning Representations*. 2020. Available at: <https://openreview.net/forum?id=HkxjqxBYDB>.